UNIVERSIDADE FEDERAL DE ALAGOAS
ENGENHARIA DE SOFTWARE - ECOM021 - 2026.2

RELATÓRIO DO PROJETO

Sistema de Aluguel de Imóveis com Microsserviços, Componentes de Software e Agente de IA

Modelagem e implementação de um sistema de gerenciamento de aluguel de apartamentos e casas.

ALUNO	Bruno Henrique Silva Rocha
MATRÍCULA	17211330
PROFESSOR	Arturo Hernández Domínguez
REPOSITÓRIO	https://github.com/3runorocha/ECOM021-eng-de-software

Novembro de 2026

Sumário

1 Enunciado do projeto baseado em microsserviços 2

Parte 1 - Modelagem 3

2 Metodologia ágil utilizada 3

3 Técnicas de reúso de software utilizadas 3

4 Desenvolvimento segundo o Scrum e as técnicas de reúso 4

- 4.1 Ponto de partida: o que reaproveitar 4
- 4.2 Planejamento e execução das sprints 4
- 4.3 Validação contínua 4
- 4.4 Revisões e o que mudou no caminho 5

5 Product Backlog e Sprint Backlog 5

- 5.1 Product Backlog 5
- 5.2 Sprint Backlog de cada sprint 7

6 Arquitetura do sistema 8

- 6.1 Arquitetura do sistema utilizando microsserviços 8
- 6.2 API Gateway 10
- 6.3 Especificação dos microsserviços e dos componentes 11
- 6.4 Sistema de recomendação 14
- 6.5 Microsserviços em execução 15

Parte 2 - Implementação 18

7 Tecnologias utilizadas 18

8 Telas do sistema, passo a passo 18

9 Código no GitHub e execução 26

10 Referências bibliográficas 27

Apêndice A - Código documentado 28

- A.1 API Gateway 29
- A.2 Microsserviço de Imóveis 35
- A.3 Microsserviço de Usuários 39
- A.4 Microsserviço de Contratos 43
- A.5 Microsserviço de Notificações 47
- A.6 Microsserviço de Recomendação 50
- A.7 Microsserviço de Agente 54
- A.8 Componentes de software e framework 62
- A.9 Aplicação derivada do framework 69

1 Enunciado do projeto baseado em microsserviços

> **Projeto (Modelagem) e Implementação de um Sistema Usando Microservices e Componentes de Software [e Agentes AI]**
>
> Obrigatoriamente deve ser um sistema usando Microservices e Componentes de Software e opcionalmente também pode usar Agentic AI.
>
> Exemplos:
>
> - Um Sistema de e-commerce Usando Microservices e Componentes de Software [e AgentesAI]
> - Um Jogo Digital não Violento Usando Microservices e Componentes de Software [e AgentesAI]
> - Um AVA (Ambiente Virtual de Aprendizagem) Usando Microservices e Componentes de Software [e AgentesAI]
> - Um Sistema de Gerenciamento de Formaturas Usando Microservices e Componentes de Software [e AgentesAI]
> - **Um Sistema de Gerenciamento de Aluguel de Apartamentos e Casas Usando Microservices e Componentes de Software [e AgentesAI]**
> - Um Sistema de Gerenciamento Eventos Científicos Usando Microservices e Componentes de Software [e AgentesAI]
>
> Observação: [ ] opcional. Linguagem de implementação sugerida: Python.

Entre os temas sugeridos, foi escolhido o **Sistema de Gerenciamento de Aluguel de Apartamentos e Casas**, incluindo a parte opcional de agente de IA. O sistema gerencia o portfólio de imóveis de uma imobiliária, o cadastro e a autenticação de usuários, os contratos de locação com cálculo de multa por atraso, alertas de vencimento, recomendação de imóveis pelo histórico do inquilino e uma busca em linguagem natural feita por um agente.

O ponto de partida foi o projeto do semestre anterior, uma biblioteca online construída para as disciplinas de Reúso de Software e Tópicos em Engenharia de Software, que já tinha API Gateway, microsserviços em Python e um framework de componentes. Adaptar esse projeto ao novo domínio é, em si, a principal aplicação de reúso deste trabalho (seção 4).

Parte 1 - Modelagem

2 Metodologia ágil utilizada

Metodologia: **Scrum**, adaptado para um desenvolvedor. O mesmo autor acumulou os papéis de Product Owner (prioriza o backlog), Scrum Master (cuida do processo e do cronograma) e time de desenvolvimento. Os eventos foram mantidos em forma enxuta:

- **Sprint:** uma por fase do plano de execução, com meta única e histórias do Product Backlog.
- **Planejamento:** o trabalho foi dividido em 16 blocos de 3 dias, de 21/09 a 07/11/2026, com a entrega em 09/11.
- **Revisão:** ao fim de cada sprint, o validador do projeto (`validar.py`) roda todas as checagens acumuladas; a sprint só fecha com tudo passando.
- **Retrospectiva:** registrada por escrito em `VALIDACAO.md` e `scrum.md`, com o que mudou no plano e por quê.

Definição de pronto. Uma história só é dada como pronta quando: (1) o comportamento está implementado e o validador passa inteiro; (2) a regra de negócio nova tem teste que falha se ela for removida; (3) a tela correspondente mostra dado real ou mostra o erro, nunca dado inventado; (4) o que ficou de fora está escrito, com o motivo.

3 Técnicas de reúso de software utilizadas

Técnicas de reúso: **Microsserviços e Componentes de Software**, complementadas por um framework de aplicação com o padrão Template Method e pelo reúso sistemático do projeto anterior.

TÉCNICA	COMO APARECE NO SISTEMA
Microsserviços	Seis serviços independentes atrás de um API Gateway, cada um com processo, porta e banco próprios. Um serviço reusa a capacidade de outro pela API, nunca pelo banco: Contratos usa Imóveis para ocupar e liberar o imóvel; Notificações e Recomendação leem Contratos.
Componentes de software	Camada `framework/` com interfaces explícitas (`IComponente*`) e ciclo de vida próprio, e implementações que falam HTTP com os serviços (`Componente*HTTP`). Quem consome não monta requisição: chama um método.
Framework com Template Method	`FrameworkBiblioteca.executar_operacao()` fixa o algoritmo (inicializar, validar, pré-processar, executar, pós-processar, registrar log, finalizar) e deixa dois hotspots obrigatórios (`configurar_componentes` e `executar_logica`). Três aplicações foram derivadas dele.
Reúso do projeto anterior	Gateway, serviço de usuários, framework e o esqueleto da interface vieram da biblioteca online. Os componentes viraram as ferramentas do agente de IA, sem cliente paralelo.
Reúso de bibliotecas	FastAPI, Pydantic, httpx, SQLite, React, SDKs do Google (Gemini) e da Anthropic, e Playwright.

4 Desenvolvimento segundo o Scrum e as técnicas de reúso

4.1 Ponto de partida: o que reaproveitar

A primeira atividade foi avaliar o projeto anterior peça por peça. O critério foi o quanto cada peça dependia do domínio de biblioteca: o que não dependia foi reusado como estava; o que dependia foi adaptado; o resto foi escrito do zero.

SITUAÇÃO	PEÇAS
Reusado sem mudança de domínio	API Gateway (roteamento e autenticação), serviço de Usuários, framework Template Method, script de subida dos serviços, padrão de camadas dos serviços (rotas → serviço → repositório → banco)
Adaptado	Catálogo de livros → Imóveis; Empréstimos → Contratos (prazo em meses e multa proporcional); Notificações e Recomendação remapeadas para o novo domínio; componentes renomeados; interface React rebrandeada
Novo	Serviço de Agente e `IComponenteAgente`; regra de um contrato aberto por imóvel com compensação; filtros no serviço; aplicação de aviso de vencimento; testes automatizados; suíte de ponta a ponta

O reúso também apareceu no planejamento: as histórias US04, US05, US15 e US16 (usuários e gateway) já vieram prontas e atravessaram a migração sem mudança.

4.2 Planejamento e execução das sprints

O plano de execução dividiu o trabalho em seis fases, cada uma virando uma sprint. O cronograma previu 16 blocos de 3 dias; o trabalho andou mais rápido que o previsto, e o tempo ganho virou uma sétima sprint de revisão e testes. A tabela mostra o período planejado e a data em que cada sprint de fato fechou.

SPRINT	META	PLANEJADO	CONCLUÍDA	PONTOS
1	Sanear o esqueleto herdado e torná-lo portável	21/09 – 26/09	21/09	11
2	Domínio: portfólio de imóveis	27/09 – 05/10	27/09	11
3	Domínio: contratos, multa e a regra do imóvel único	06/10 – 17/10	27/09	20
4	Interface web que mostra só dados reais	18/10 – 26/10	07/10	19
5	Agente de IA	27/10 – 01/11	10/10	13
6	Qualidade e documentação	02/11 – 07/11	13/10	8
7	Revisão de código e testes de ponta a ponta	folga dos blocos 15 e 16	28/10	-

4.3 Validação contínua

Desde a primeira sprint, cada comportamento entregue ganhou uma checagem no `validar.py`, que sobe os sete processos com bancos temporários e testa tudo pelo gateway. As checagens

são cumulativas: uma sprint só fecha com as dela e as de todas as anteriores passando. Ao final, o projeto tem:

VERIFICAÇÃO	QUANTIDADE	RESULTADO
Checagens do validador (estáticas e com o sistema no ar)	54	todas passando
Testes de unidade (regras de negócio, com repositórios fake)	139	todos passando
Testes de integração pelo gateway	5	todos passando
Casos de ponta a ponta pelo navegador (Playwright)	29	27 passando, 2 defeitos registrados

4.4 Revisões e o que mudou no caminho

- **Renomear entidade é atômico.** Trocar Livro por Imóvel quebrava os componentes que consumiam o serviço; as sprints planejadas para isso foram fundidas.
- **Nenhuma tela mente.** Três telas herdadas mostravam dados inventados. Foram reescritas para ler os serviços e mostrar o erro quando o serviço cai.
- **Testes acham o que a revisão não acha.** A suíte de unidade revelou um erro de fuso que dava ao token 11 horas de validade em vez de 8.
- **Revisão de código.** Encontrou senha com hash fraco, segredo do token no código, qualquer usuário lendo contrato alheio e imóveis sem restrição de perfil. Tudo corrigido com teste e checagem.
- **Defeitos em aberto.** A suíte de ponta a ponta registrou dois: o serviço de recomendação não confere quem está pedindo, e no celular o menu lateral espreme o conteúdo.

5 Product Backlog e Sprint Backlog

5.1 Product Backlog

Atores: **inquilino** (procura imóvel, assina e encerra contrato, recebe avisos), **proprietário/corretor** (administra o portfólio), **sistema** (tarefas automáticas) e **desenvolvedor**. Total: 23 histórias, 103 pontos.

ID	COMO…	QUERO…	PARA…	PRIORIDADE	PTS
US01	corretor	cadastrar imóveis no portfólio	manter a oferta atualizada	Alta	3
US02	inquilino	filtrar imóveis por cidade, tipo, quartos e faixa de aluguel	achar o que cabe no meu bolso	Alta	5
US03	sistema	tratar disponibilidade como sim/não	impedir dois contratos no mesmo imóvel	Alta	3
US04	inquilino	me cadastrar	ter acesso ao sistema	Alta	2
US05	inquilino	autenticar com e-mail e senha	usar o sistema com segurança	Alta	3

ID	COMO…	QUERO…	PARA…	PRIORIDADE	PTS
US06	corretor	listar os usuários e seus contratos em vigor	administrar a base	Média	3
US07	inquilino	assinar um contrato de aluguel	alugar o imóvel	Alta	5
US08	inquilino	encerrar o contrato	devolver o imóvel	Alta	3
US09	sistema	cobrar multa proporcional ao aluguel por dia de atraso	aplicar a política da imobiliária	Alta	5
US10	sistema	recusar segundo contrato em imóvel ocupado	manter a realidade consistente	Alta	5
US11	sistema	marcar contratos vencidos como atrasados	manter o status correto	Média	2
US12	inquilino	receber alertas de vencimento e atraso	não perder o prazo	Média	3
US13	sistema	varrer os contratos e gerar alertas sozinho	notificar sem ação manual	Média	3
US14	inquilino	receber recomendações de imóveis pelo meu histórico	descobrir opções parecidas	Baixa	5
US15	sistema	rotear tudo por um API Gateway	ter ponto de entrada único	Alta	5
US16	sistema	validar o token no gateway	proteger as rotas	Alta	3
US17	desenvolvedor	estruturar em microsserviços independentes	facilitar manutenção e evolução	Alta	8
US18	inquilino	usar o sistema por uma interface web	fazer tudo de forma visual	Alta	5
US19	corretor	ver a saúde real dos microsserviços	saber o que está no ar	Baixa	3
US20	desenvolvedor	reusar um framework com Template Method e hotspots	criar aplicações novas rápido	Média	8
US21	inquilino	buscar imóvel em linguagem natural	pedir do meu jeito, sem preencher filtro	Baixa	8
US22	sistema	deixar o agente redigir o aviso de vencimento	mandar um texto melhor que o padrão	Baixa	5
US23	desenvolvedor	ter testes automatizados da regra de negócio	mexer no código sem medo	Média	8

5.2 Sprint Backlog de cada sprint

Sprint 1 - Sanear o esqueleto herdado e torná-lo portável

`Histórias: US17, US19 · 11 pontos · concluída em 21/09`

- Corrigir o hotspot obrigatório do framework, que não era obrigatório de fato
- Externalizar 17 URLs e 5 caminhos de banco em variável de ambiente
- Trocar `@app.on_event` depreciado por `lifespan`
- Filtrar headers hop-by-hop no proxy do gateway
- Criar o script que sobe todos os serviços num comando (`subir_servicos.py`)

Revisão: O `PUBLIC_ROUTES` que parecia bug não era: o gateway consome o primeiro segmento da rota como seletor de serviço. Descartado após verificação.

Sprint 2 - Domínio: portfólio de imóveis

`Histórias: US01, US02, US03 · 11 pontos · concluída em 27/09`

- `Livro` → `Imovel`: tipo, endereço, cidade, quartos, banheiros, área, aluguel
- `atualizar_disponibilidade(delta)` → `definir_disponibilidade(bool)`
- Filtros de cidade, tipo, quartos mínimos e faixa de valor, aplicados no serviço
- Seeds com 18 imóveis de Maceió, Recife e João Pessoa

Revisão: Renomear o serviço arrastou os componentes que o consomem; não dava para separar em sprints diferentes sem deixar o repositório incoerente.

Sprint 3 - Domínio: contratos, multa e a regra do imóvel único

`Histórias: US07 a US11 · 20 pontos · concluída em 27/09`

- `Emprestimo` → `Contrato`, com `valor_mensal` gravado na assinatura
- Prazo em meses, com ajuste de mês curto (31/01 + 1 mês = 28/02)
- Multa de 1/30 do aluguel por dia de atraso, no lugar dos R$ 0,50 fixos
- Um contrato aberto por imóvel, imposto no serviço e por índice no banco
- Compensação: se o INSERT falha, o imóvel volta a ficar disponível

Revisão: Renomear entidade é operação atômica: as quatro sprints planejadas para isso viraram uma.

Sprint 4 - Interface web que mostra só dados reais

`Histórias: US06, US12, US14, US18, US19 · 19 pontos · concluída em 07/10`

- Telas de imóveis, contratos, usuários, notificações e recomendação com dados reais
- Status dos serviços no menu lendo o `/health` de verdade
- Estado de erro em toda tela: falha de backend aparece como falha

Revisão: Três telas (Dashboard, Notificações, Usuários) não estavam em nenhuma história e mostravam dados inventados. Entraram como US06, US12 e US19 revisadas.

Sprint 5 - Agente de IA

`Histórias: US21, US22 · 13 pontos · concluída em 10/10`

- Serviço de agente em :8006, com três ferramentas de leitura
- `IComponenteAgente`: o agente consome componentes e também é um componente
- `AppAvisoVencimento` orquestra o agente pelo Template Method
- Degradação para texto padrão quando o agente está fora do ar ou sem credencial

Revisão: A qualidade da resposta do modelo não pôde ser verificada: não há chave da API da Anthropic no ambiente. Todo o resto foi testado.

Sprint 6 - Qualidade e documentação

`Histórias: US23 · 8 pontos · concluída em 13/10`

- Testes de unidade com repositórios fake e testes de integração pelo gateway
- Suíte integrada ao validador do projeto (`validar.py`)
- Diagramas reescritos em Mermaid, refletindo o código final
- README, guia do framework e backlog atualizados

Revisão: A suíte achou um bug de fuso no token (declarava 11 horas de validade em vez de 8) que nenhuma checagem anterior pegava.

Sprint 7 - Revisão de código e testes de ponta a ponta

`Histórias: US05, US16, US23 (refinadas) · sem pontos novos · concluída em 28/10`

- Revisão de código arquivo por arquivo
- Senha com PBKDF2-SHA256 (600 mil iterações, sal por usuário) e segredo do token fora do código
- Gateway repassa a identidade aos serviços; contratos restritos ao dono
- Imóveis restritos a admin e proprietário; cadastro público cria só inquilino
- `/health` em paralelo, encerramento com compensação, e-mail repetido devolvendo 400
- Suíte de 29 casos de ponta a ponta com Playwright e demonstração gravada
- Agente com dois provedores de modelo: Gemini (nível gratuito) ou Claude

Revisão: Sprint extra, aberta com a folga do cronograma. Sem pontos novos: refinou histórias já entregues. A suíte de ponta a ponta encontrou dois defeitos que ficaram registrados (seção 4.4).

6 Arquitetura do sistema

6.1 Arquitetura do sistema utilizando microsserviços

O sistema roda em sete processos. Toda requisição do navegador entra pelo API Gateway, que autentica e encaminha ao microsserviço responsável. Cada serviço tem banco SQLite próprio e não acessa o banco de outro: quando precisa de dado alheio, chama o serviço dono pela API. O agente é o único que sai para a internet, para a API do modelo: Gemini ou Claude.



QUEM CHAMA		SERVIÇO CHAMADO	PARA QUÊ
Gateway	→	Usuários	valida o token de cada requisição
Contratos	→	Imóveis	ocupa e libera o imóvel
Notificações	→	Contratos	lê os contratos na varredura
Recomendação	→	Contratos e Imóveis	lê o histórico e o portfólio
Agente	→	Imóveis e Contratos	ferramentas, pela camada de componentes

Figura 1 - Arquitetura de microsserviços: gateway, serviços, bancos e chamadas entre serviços

PROCESSO	PORTA	BANCO	RESPONSABILIDADE
API Gateway	8000	-	Ponto de entrada único: roteamento, autenticação, identidade, saúde
Imóveis	8001	imoveis.db	Portfólio, filtros e disponibilidade
Usuários	8002	usuarios.db	Cadastro, login, token e perfis
Contratos	8003	contratos.db	Locação, encerramento e multa
Notificações	8004	notificacoes.db	Alertas de prazo e atraso
Recomendação	8005	recomendacao.db	Sugestões pelo histórico do inquilino
Agente	8006	-	Busca em linguagem natural e redação de avisos
Interface web	5173	-	React, consome só o gateway

A operação central, assinar um contrato, exige que dois serviços concordem sobre o mesmo fato: o imóvel está ocupado. A Figura 2 mostra a sequência e a compensação usada quando a grava-

ção falha depois de ocupar o imóvel.

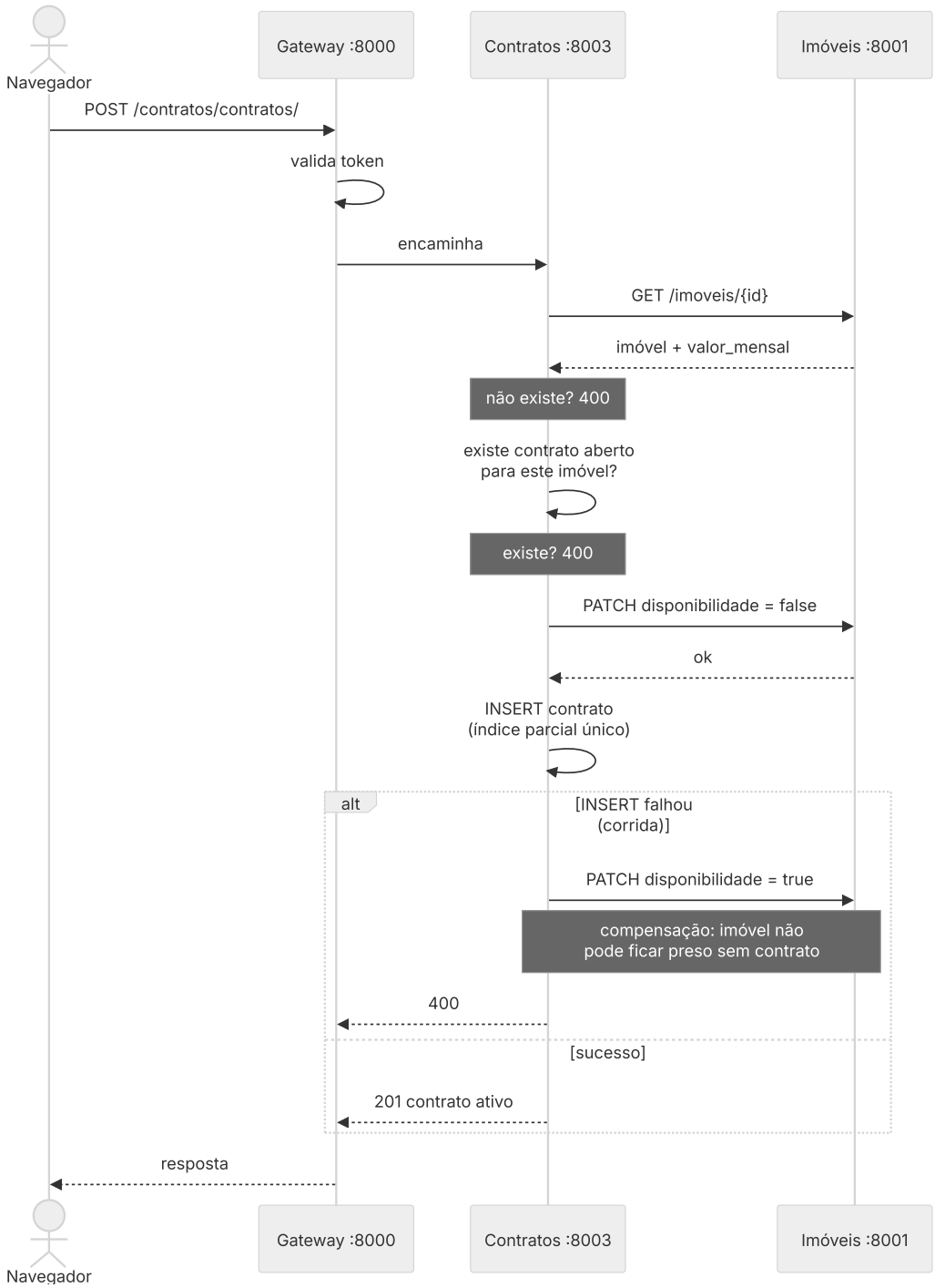


Figura 2 - Assinatura de contrato entre Gateway, Contratos e Imóveis

6.2 API Gateway

O gateway (`biblioteca/gateway/`, porta 8000) é o único caminho de fora para os serviços. Ele não tem banco nem regra de negócio. Responsabilidades:

- **Roteamento:** `/{servico}/{caminho}` é encaminhado ao serviço correspondente, com os métodos GET, POST, PATCH, PUT e DELETE.
- **Autenticação:** toda rota exige o token, validado no serviço de Usuários (`POST /usuarios/validar-token`). Rotas públicas: registro e login. Rotas livres: `/health`, `/services` e a

documentação.

- **Identidade:** depois de validar, injeta `x-usuario-id` e `x-usuario-tipo` na requisição repassada, descartando headers com esses nomes vindos do cliente. Os serviços usam esses dados para restringir por perfil.
- **Saúde:** `GET /health` consulta os seis serviços em paralelo; `GET /services` lista portas e descrições.
- **Proxy limpo:** filtra headers hop-by-hop e usa um cliente HTTP compartilhado que não guarda cookies.

ROTA	ACESSO	DESTINO
`/imoveis/*`, `/usuarios/*`, `/contratos/*`	token	Imóveis, Usuários, Contratos
`/notificacoes/*`, `/recomendacao/*`, `/agente/*`	token	Notificações, Recomendação, Agente
`POST /usuarios/usuarios/registro` e `/login`	pública	Usuários
`GET /health`, `GET /services`, `/docs`	livre	o próprio gateway

O código documentado do gateway está no Apêndice A.1.

6.3 Especificação dos microsserviços e dos componentes

Todos os serviços seguem as mesmas camadas: `routes.py` (HTTP) → `service.py` (regra de negócio) → `repository.py` (SQL) → `database.py` (conexão e esquema), com modelos Pydantic em `models.py`. Chamadas sem os headers de identidade vêm de outro serviço e não são restringidas.

Imóveis `:8001 · imoveis.db`

Portfólio e disponibilidade. Imóvel: título, tipo (apartamento ou casa), endereço, cidade, quartos, banheiros, área, aluguel e situação.

MÉTODO	ROTA	DESCRIÇÃO
GET	`/imoveis/`	Lista com filtros de cidade (ignora acento e maiúsculas), tipo, quartos mínimos, faixa de aluguel e situação
GET	`/imoveis/{id}`	Detalhe de um imóvel
POST	`/imoveis/`	Cadastra; o imóvel nasce disponível. Só admin e proprietário
PATCH	`/imoveis/{id}`	Altera dados. Só admin e proprietário
DELETE	`/imoveis/{id}`	Remove; recusa imóvel alugado. Só admin e proprietário
PATCH	`/imoveis/{id}/disponibilidade`	Ocupa ou libera. Admin ou chamada interna do serviço de Contratos

Código: Apêndice A.2.

Usuários `:8002 · usuarios.db`

Cadastro, autenticação e perfis (inquilino, proprietário, admin). Senha com PBKDF2-SHA256, 600 mil iterações e sal por usuário; token assinado com HMAC, válido por 8 horas.

MÉTODO	ROTA	DESCRIÇÃO
POST	`/usuarios/registro`	Cadastro público: cria só inquilino (o primeiro usuário pode nascer admin)
POST	`/usuarios/login`	Devolve o token e os dados do usuário
GET	`/usuarios/`	Lista os usuários
GET	`/usuarios/{id}`	Detalhe de um usuário
PATCH	`/usuarios/{id}`	Altera dados; o próprio usuário ou admin. Perfil e situação, só admin
POST	`/usuarios/validar-token`	Usada pelo gateway: confere o token e se o usuário segue ativo

Código: Apêndice A.3.

Contratos `:8003 · contratos.db`

Locação com prazo padrão de 12 meses, valor do aluguel gravado na assinatura e multa de 1/30 do aluguel por dia de atraso. Um contrato aberto por imóvel, garantido no serviço e por índice único parcial no banco.

MÉTODO	ROTA	DESCRIÇÃO
POST	`/contratos/`	Assina: ocupa o imóvel no serviço de Imóveis e grava; desfaz a ocupação se a gravação falhar. Inquilino só assina para si
POST	`/contratos/{id}/encerrar`	Encerra, calcula a multa e libera o imóvel, com compensação se a gravação falhar
GET	`/contratos/`	Lista e marca os vencidos como atrasados; inquilino vê só os seus
GET	`/contratos/{id}`	Detalhe; contrato de outro inquilino responde 404

Código: Apêndice A.4.

Notificações `:8004 · notificacoes.db`

Alertas por usuário. A varredura lê os contratos e gera alerta de atraso (com a multa acumulada) e de prazo próximo (até 2 dias), sem repetir aviso não lido igual.

MÉTODO	ROTA	DESCRIÇÃO
POST	`/notificacoes/`	Cria uma notificação
GET	`/notificacoes/usuario/{id}`	Notificações de um usuário
PATCH	`/notificacoes/{id}/ler`	Marca como lida
PATCH	`/notificacoes/usuario/{id}/ler-todas`	Marca todas como lidas
POST	`/notificacoes/varredura`	Varre os contratos e gera os alertas

Código: Apêndice A.5.

Agente `:8006 · -`

Busca em linguagem natural e redação de avisos com o Gemini (Google, com nível gratuito) ou o Claude (Anthropic), escolhidos em provedores.py. As ferramentas do modelo são os próprios componentes do framework, todas de leitura: buscar_imoveis, detalhar_imovel e listar_contratos (esta restrita aos contratos de quem pergunta). Sem chave da API, sobe normalmente e responde 503 nas rotas que chamam o modelo.

MÉTODO	ROTA	DESCRIÇÃO
POST	`/agente/perguntar`	Responde a pergunta usando as ferramentas
POST	`/agente/redigir-aviso`	Redige o texto do aviso a partir de fatos já calculados
GET	`/agente/status`	Se há credencial, qual modelo e quais ferramentas

Código: Apêndice A.7.

O serviço de Recomendação é especificado na seção 6.4.

Componentes de software

A camada de componentes (`biblioteca/framework/`) define contratos explícitos e implementações HTTP. Todo componente tem `inicializar()` e `finalizar()`, onde o cliente HTTP abre e fecha; quem controla o ciclo é o framework, nas aplicações, ou o `lifespan` do FastAPI, no serviço de agente.

INTERFACE	OPERAÇÕES	IMPLEMENTAÇÃO
`IComponente`	inicializar, finalizar, get_nome	base de todas
`IComponenteImovel`	buscar_imoveis, cadastrar_imovel, definir_disponibilidade	`ComponenteImovelHTTP`
`IComponenteUsuario`	registrar, autenticar, buscar_usuario	`ComponenteUsuarioHTTP`
`IComponenteContrato`	registrar_contrato, encerrar_contrato, listar_contratos	`ComponenteContratoHTTP`
`IComponenteNotificacao`	enviar, listar	`ComponenteNotificacaoHTTP`
`IComponenteRecomendacao`	recomendar, obter_perfil	`ComponenteRecomendacaoHTTP`
`IComponenteAgente`	perguntar, redigir_aviso	`ComponenteAgenteHTTP`

O framework `FrameworkBiblioteca` aplica o Template Method sobre esses componentes. Três aplicações foram derivadas: `AppContrato` (assina e notifica), `AppRecomendacao` (perfil e recomendações) e `AppAvisoVencimento` (levanta contratos vencendo, pede ao agente o texto e envia a notificação). As ferramentas do agente usam a mesma camada, então o agente não é um cliente paralelo.

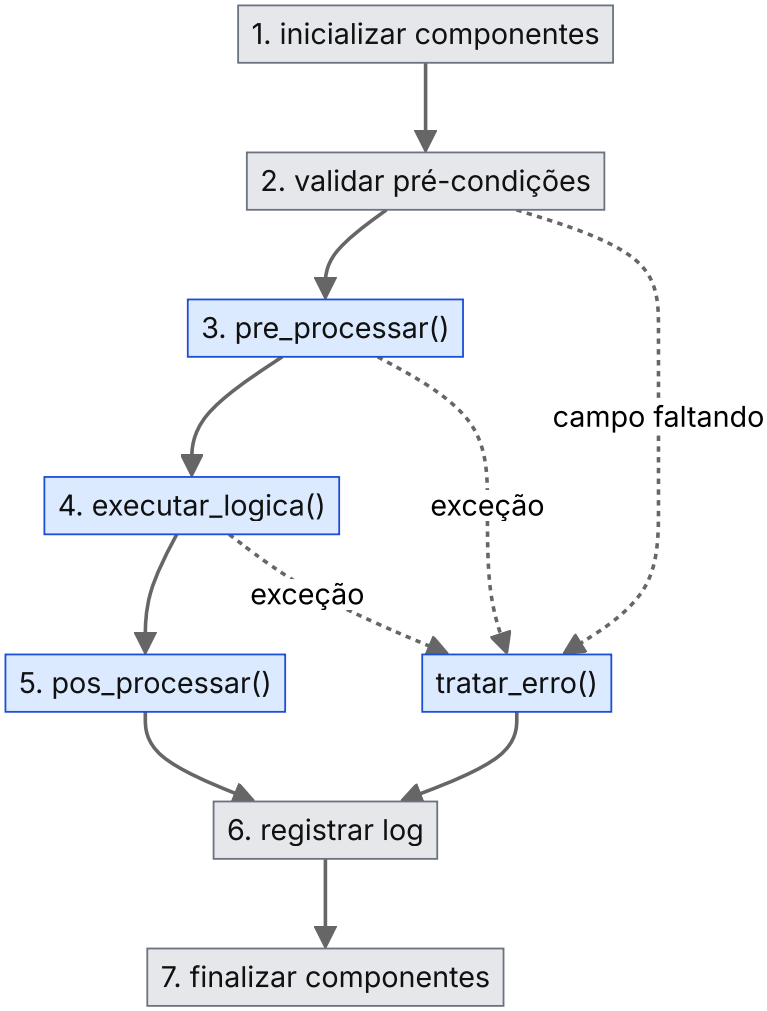


Figura 3 - Algoritmo fixo do Template Method (cinza) e hotspots (azul)

Código: Apêndice A.8 (componentes e framework) e A.9 (aplicação derivada).

6.4 Sistema de recomendação

O serviço de Recomendação (porta 8005, banco `recomendacao.db`) sugere imóveis disponíveis a partir do histórico de contratos do inquilino. É uma recomendação baseada em conteúdo, com pontuação explicável:

1. **Perfil.** Busca o histórico no serviço de Contratos e o portfólio no serviço de Imóveis (uma chamada só). Conta os tipos e as cidades dos imóveis já alugados e guarda os três mais frequentes de cada.
2. **Candidatos.** Só imóveis disponíveis. O imóvel de um contrato ainda aberto da própria pessoa é excluído.
3. **Pontuação.** +3 se o tipo está entre os preferidos; +2 se a cidade está entre as preferidas; −2 se a pessoa já morou ali; −1 se o imóvel já foi recomendado antes a ela. Pontuação máxima: 5.
4. **Resultado.** Descarta pontuação zero ou negativa, ordena da maior para a menor, devolve as N primeiras (padrão 5) com o motivo em texto e registra o que foi recomendado.

MÉTODO	ROTA	DESCRIÇÃO
GET	`/recomendacao/perfil/{usuario_id}`	Perfil calculado: tipos e cidades favoritos, total de contratos
GET	`/recomendacao/{usuario_id}?limite=5`	Imóveis recomendados com pontuação e motivo

Exemplo real, com os dados de exemplo do sistema: a inquilina Ana tem histórico de apartamentos e casas em Recife e Maceió; o Studio na Jatiúca (apartamento em Maceió) recebe 5 de 5, e a Kitnet no Centro, onde ela já morou, recebe 3. A busca em linguagem natural do agente complementa a recomendação: o usuário descreve o que quer e o modelo traduz em filtros.

Código: Apêndice A.6.

6.5 Microsserviços em execução

Um comando sobe os sete processos (`python subir_servicos.py`). A saída lista cada serviço com sua porta:

```
PS C:\DevTools\ECOM021-eng-de-software> python subir_servicos.py
Subindo os servicos:
  imoveis       :8001   pid 372
  usuarios      :8002   pid 14696
  contratos     :8003   pid 32432
  notificacoes  :8004   pid 8632
  recomendacao  :8005   pid 11996
  agente        :8006   pid 34992
  gateway       :8000   pid 32024

Gateway:  http://localhost:8000
Swagger:  http://localhost:8000/docs
Health:   http://localhost:8000/health
```

Saída real do script ao subir os serviços.

```
{
  "gateway": "ok",
  "status_geral": "ok",
  "servicos": {
    "imoveis": {
      "status": "ok",
      "porta": 8001
    },
    "usuarios": {
      "status": "ok",
      "porta": 8002
    },
    "contratos": {
      "status": "ok",
      "porta": 8003
    },
    "notificacoes": {
      "status": "ok",
      "porta": 8004
    },
    "recomendacao": {
      "status": "ok",
      "porta": 8005
    },
    "agente": {
      "status": "ok",
      "porta": 8006
    }
  }
}
```


Figura 4 - `GET /health` do gateway: os seis serviços com status ok



Figura 5 - Status dos serviços no menu da interface, lido do `/health`


```
{
  "gateway": {
    "porta": 8000,
    "descricao": "API Gateway - ponto de entrada unico"
  },
  "imoveis": {
    "porta": 8001,
    "descricao": "Portfolio de apartamentos e casas"
  },
  "usuarios": {
    "porta": 8002,
    "descricao": "Registro e autenticacao de usuarios"
  },
  "contratos": {
    "porta": 8003,
    "descricao": "Locacao, encerramento e multa"
  },
  "notificacoes": {
    "porta": 8004,
    "descricao": "Alertas de prazo e atraso"
  },
  "recomendacao": {
    "porta": 8005,
    "descricao": "Recomendacao por perfil do usuario"
  },
```


Figura 6 - `GET /services`: o gateway e os seis microsserviços com porta e descrição

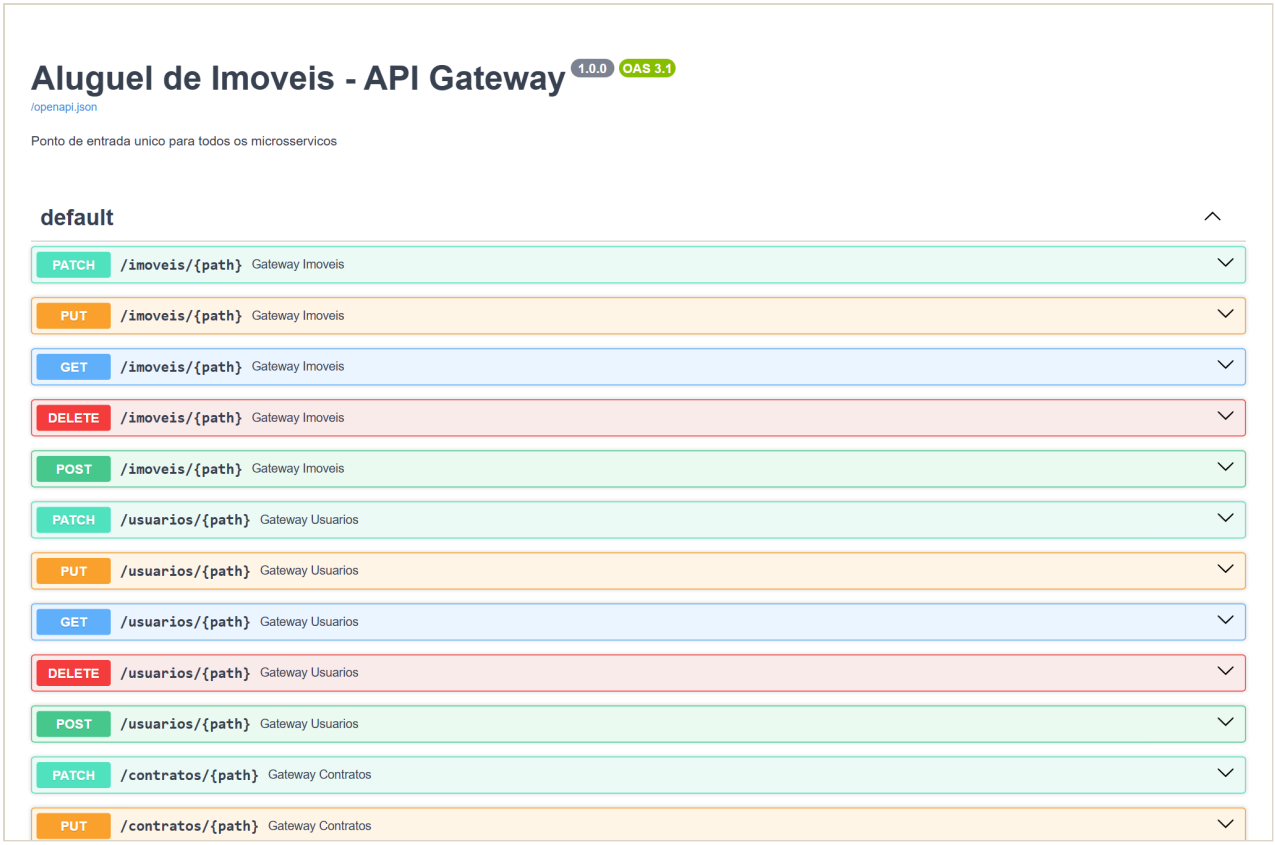


Figura 7 - Documentação da API do gateway (Swagger), com as rotas de cada microsserviço

Parte 2 - Implementação

7 Tecnologias utilizadas

MICROSSERVIÇO	LINGUAGEM	FRAMEWORK E BIBLIOTECAS	BANCO DE DADOS
API Gateway	Python 3.14	FastAPI 0.141, Uvicorn 0.52, httpx 0.28	não tem
Imóveis	Python 3.14	FastAPI, Pydantic 2.13	SQLite (imoveis.db)
Usuários	Python 3.14	FastAPI, Pydantic; hashlib/hmac (PBKDF2, HMAC)	SQLite (usuarios.db)
Contratos	Python 3.14	FastAPI, Pydantic, httpx	SQLite (contratos.db)
Notificações	Python 3.14	FastAPI, Pydantic, httpx	SQLite (notificacoes.db)
Recomendação	Python 3.14	FastAPI, Pydantic, httpx	SQLite (recomendacao.db)
Agente	Python 3.14	FastAPI; SDK google-genai 3.0 com o modelo gemini-3.8-flash, ou SDK da Anthropic 1.13 com claude-opus-5-5	não tem
Componentes e framework	Python 3.14	httpx; abc (interfaces)	-
Interface web	JavaScript	React 19, React Router 7, Axios, Vite 8	-

USO	FERRAMENTAS
Testes	pytest (unidade e integração), Playwright para Python (ponta a ponta)
Qualidade	ESLint no frontend; validador próprio do projeto (`validar.py`)
Documentação	Swagger/OpenAPI gerado pelo FastAPI; diagramas em Mermaid
Versionamento	Git e GitHub

8 Telas do sistema, passo a passo

As telas abaixo foram capturadas do sistema em funcionamento, com os dados de exemplo. Em todas, os microsserviços usados estão **destacados em vermelho**: no menu lateral, cada item mostra a porta do serviço que o atende, e no topo de cada tela aparece a chamada feita ao serviço (por exemplo, `GET :8001/imoveis`).

Passo 1 - Login

Entrada com e-mail e senha. A validação é feita pelo serviço de Usuários (:8002), passando pelo gateway. Senha errada mostra "Email ou senha inválidos".

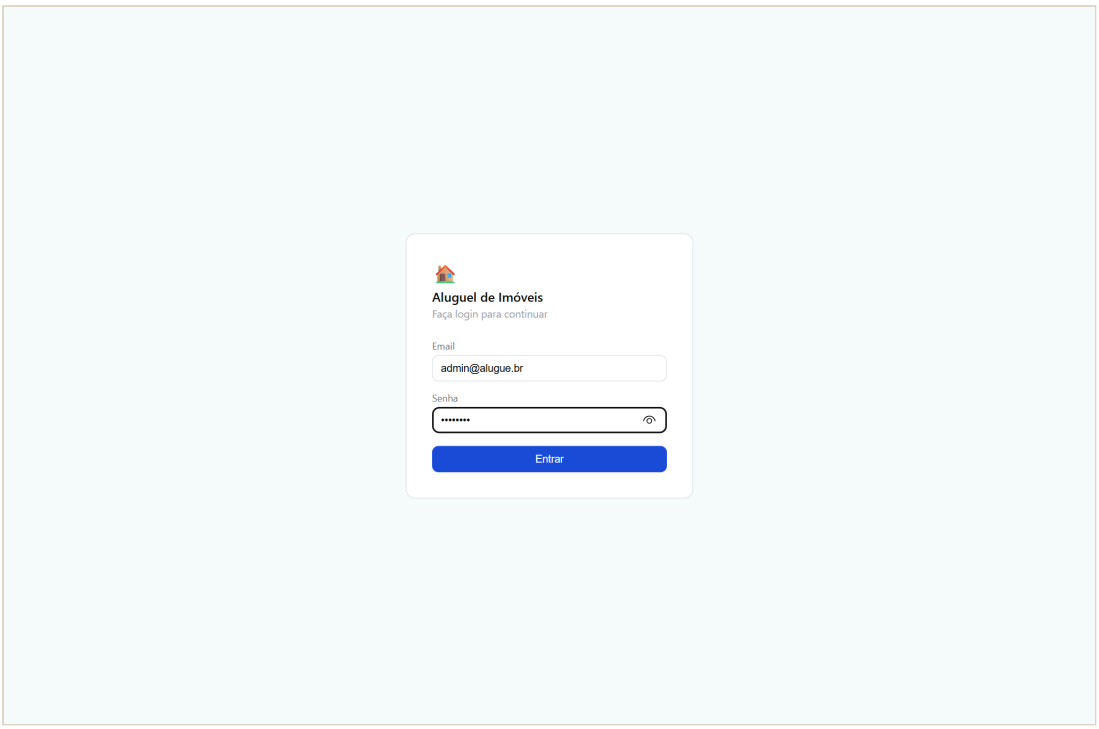


Figura 8 - Login

Passo 2 - Painel

Resumo com números reais, cada um lido do serviço dono: imóveis (:8001), contratos em vigor, atrasos e receita (:8003), usuários (:8002) e notificações (:8004). Embaixo, o status dos sete processos.

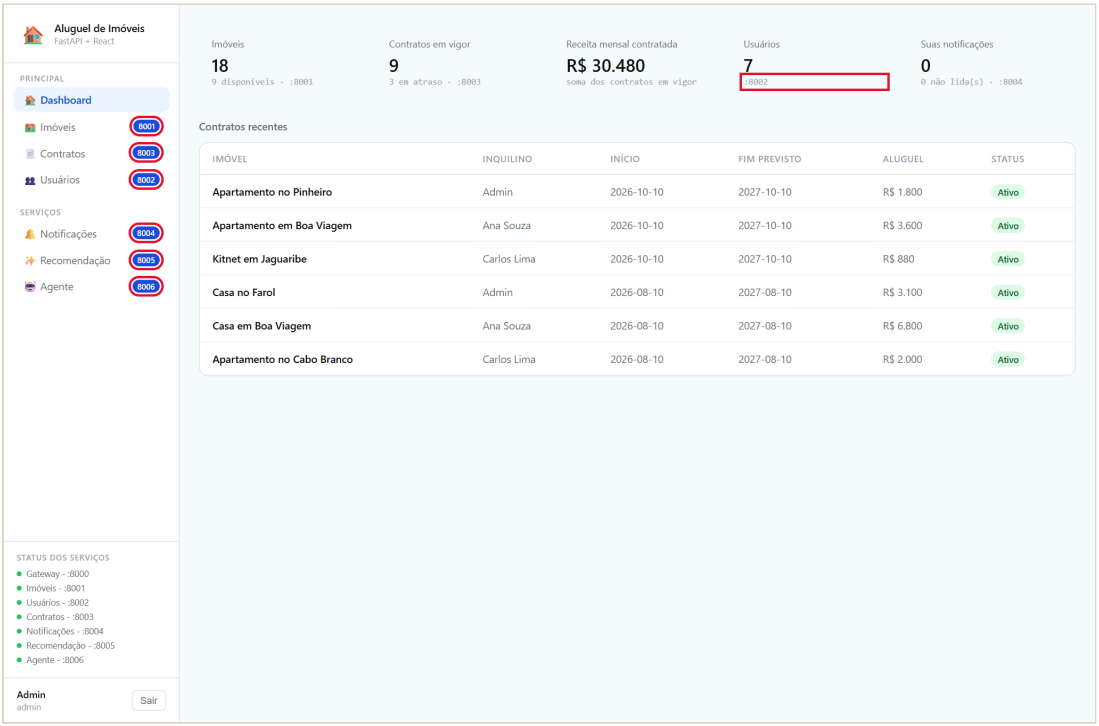


Figura 9 - Painel

Passo 3 - Catálogo de imóveis

Portfólio do serviço de Imóveis (:8001), com a situação de cada imóvel. A busca procura no título e no endereço.

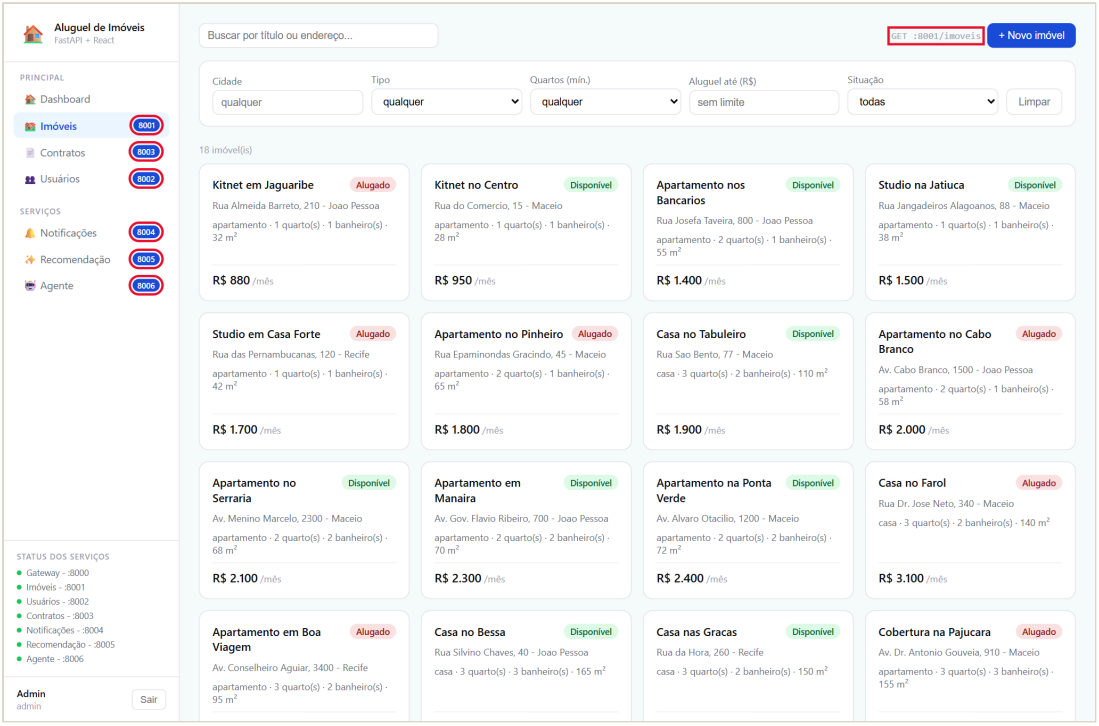


Figura 10 - Catálogo de imóveis

Passo 4 - Filtros combinados

Cidade "Maceió", tipo casa e situação disponível: os filtros são aplicados no próprio serviço de Imóveis, e o de cidade ignora acentos e maiúsculas.

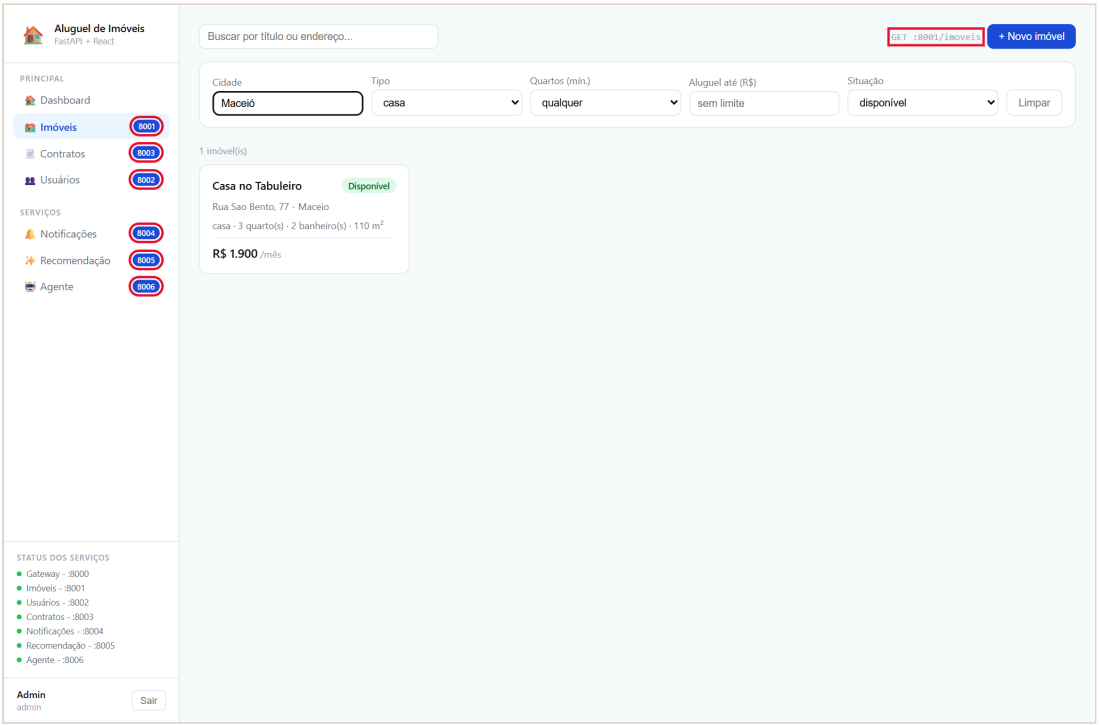


Figura 11 - Filtros combinados

Passo 5 - Cadastro de imóvel

Formulário de novo imóvel, disponível só para admin e proprietário. Campos obrigatórios em branco são recusados.

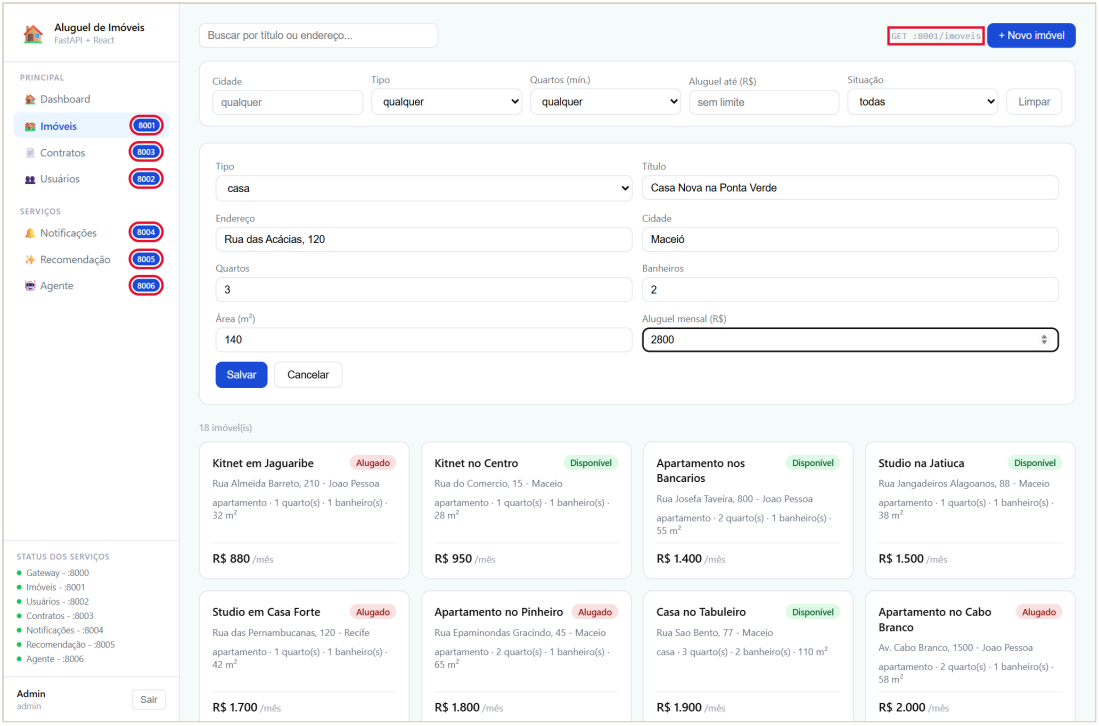


Figura 12 - Cadastro de imóvel

Passo 6 - Contratos

Todos os contratos (:8003) para o admin, com datas, aluguel, multa e situação. Contratos vencidos aparecem como atrasados, com os dias desde o vencimento.

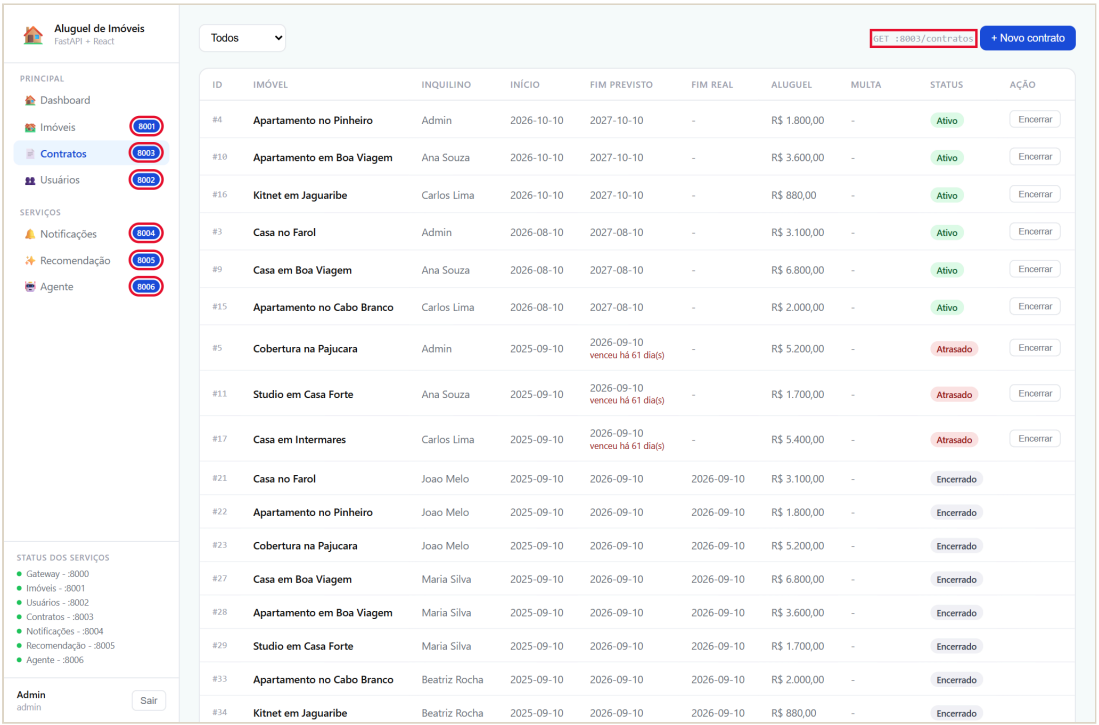


Figura 13 - Contratos

Passo 7 - Novo contrato

O formulário oferece só imóveis sem contrato em aberto. Ao registrar, o serviço de Contratos ocupa o imóvel no serviço de Imóveis.

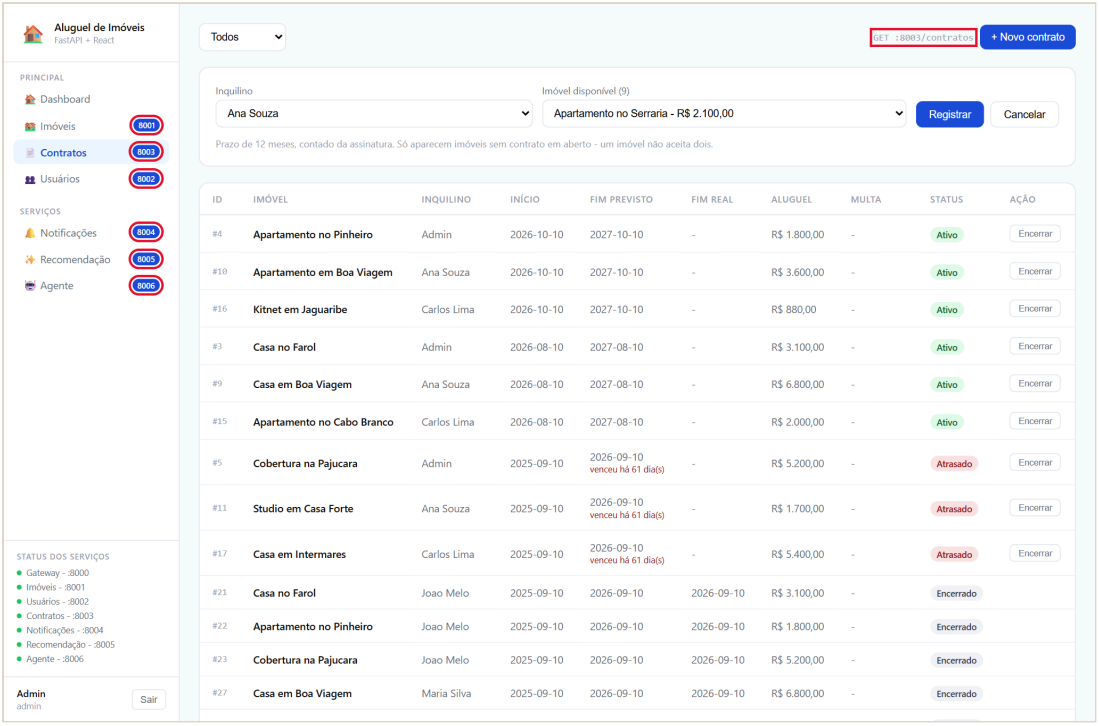


Figura 14 - Novo contrato

Passo 8 - Contrato registrado

O contrato da Ana no Apartamento no Serraria entra como ativo, com prazo de 12 meses. O imóvel sai da lista de disponíveis.

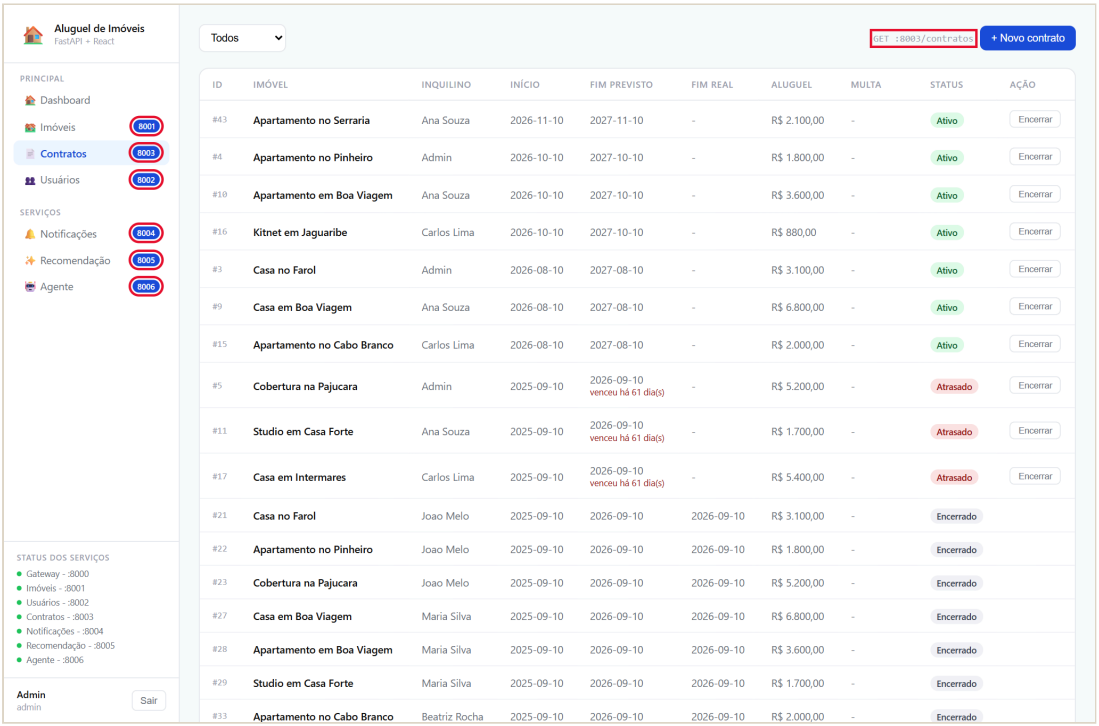


Figura 15 - Contrato registrado

Passo 9 - Filtro de atrasados

Só os contratos atrasados. A multa é de 1/30 do aluguel por dia de atraso, calculada no encerramento.

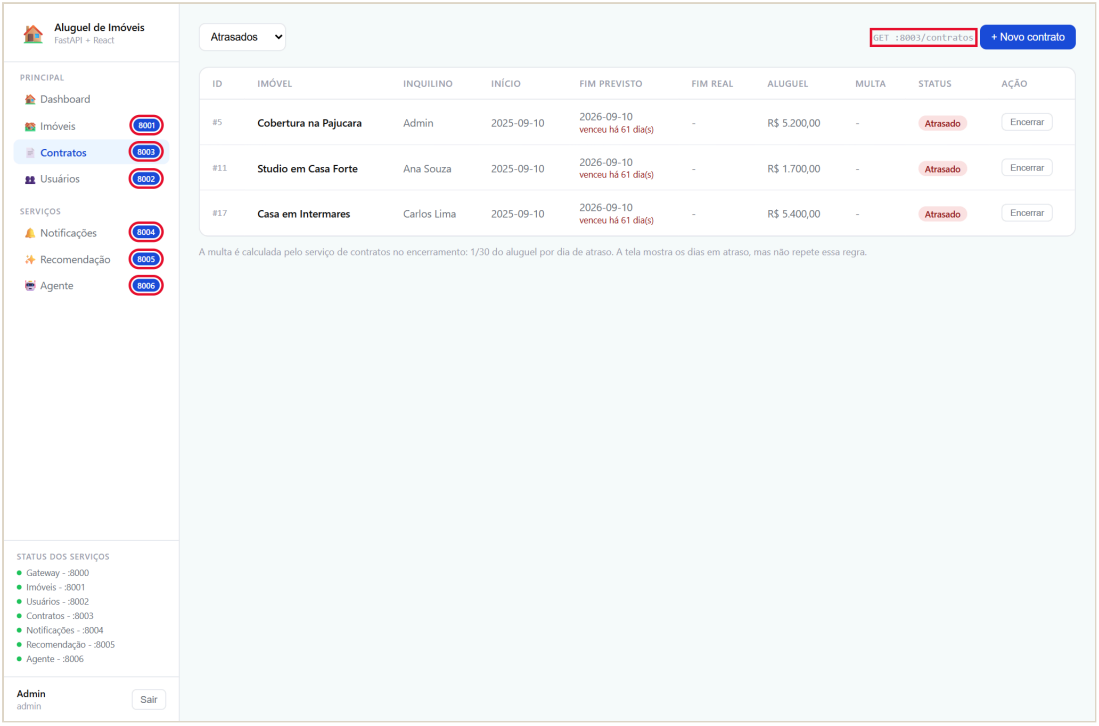


Figura 16 - Filtro de atrasados

Passo 10 - Usuários

Lista do serviço de Usuários (:8002) com perfil e contratos em vigor. O admin muda o perfil pelo seletor; o cadastro público cria só inquilinos.

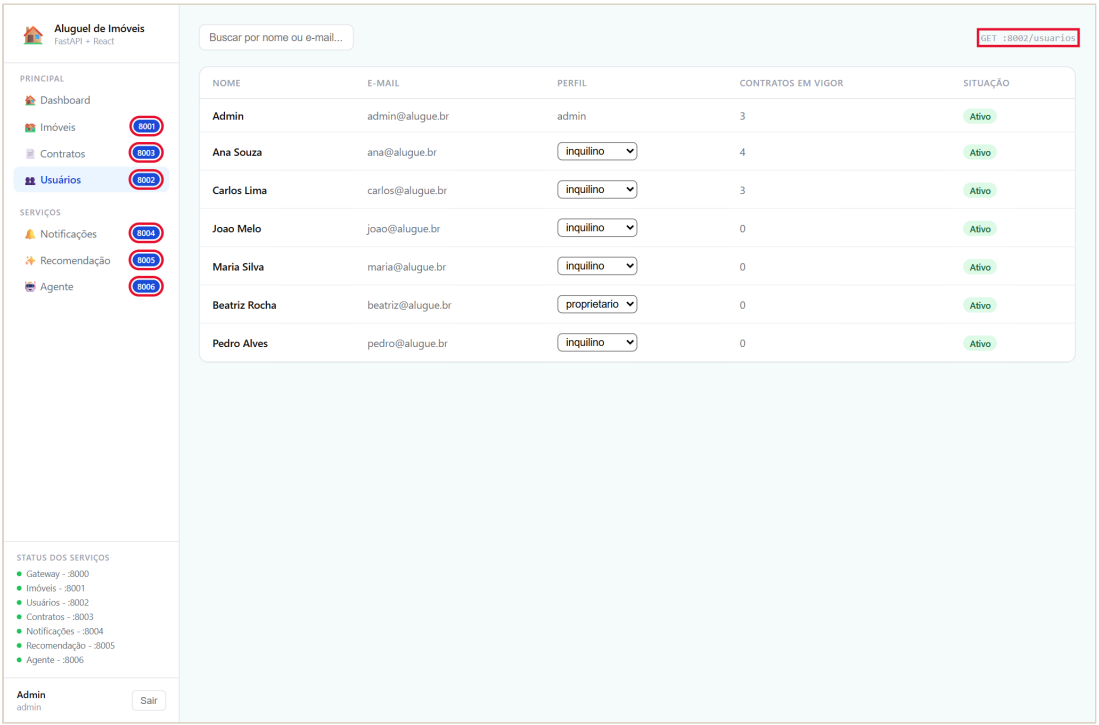


Figura 17 - Usuários

Passo 11 - Notificações

A varredura do serviço de Notificações (:8004) lê os contratos e gera o alerta de atraso com a multa acumulada.

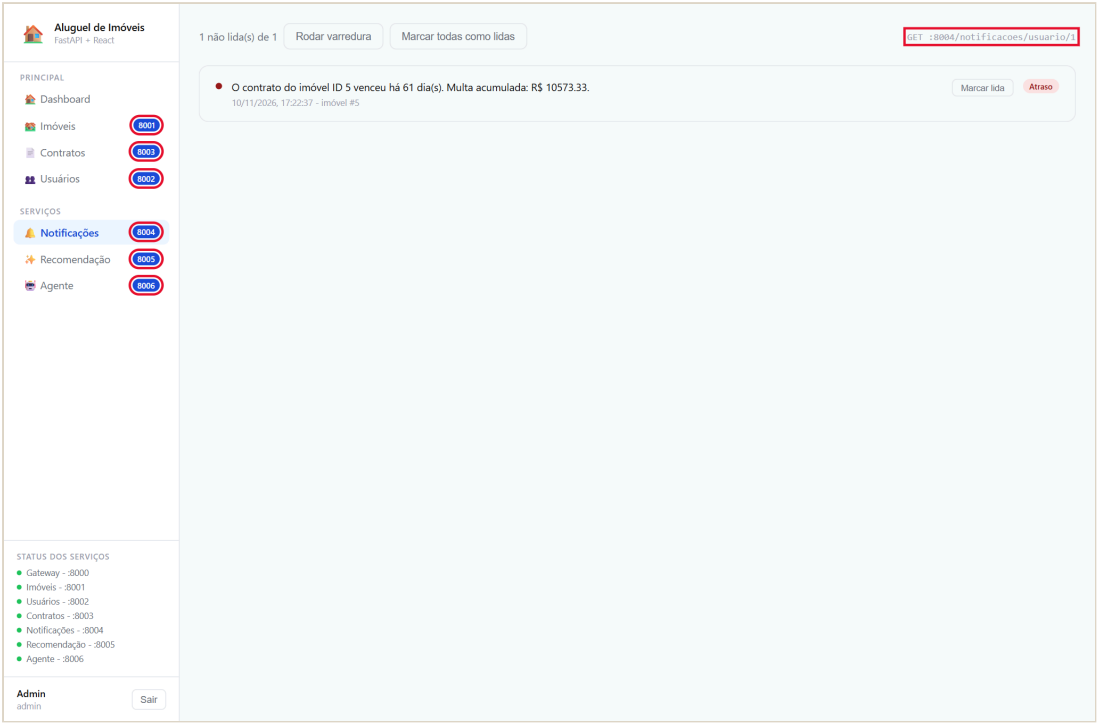


Figura 18 - Notificações

Passo 12 - Recomendação

Perfil da Ana calculado pelo serviço de Recomendação (:8005) a partir do histórico, e os imóveis pontuados com o motivo.

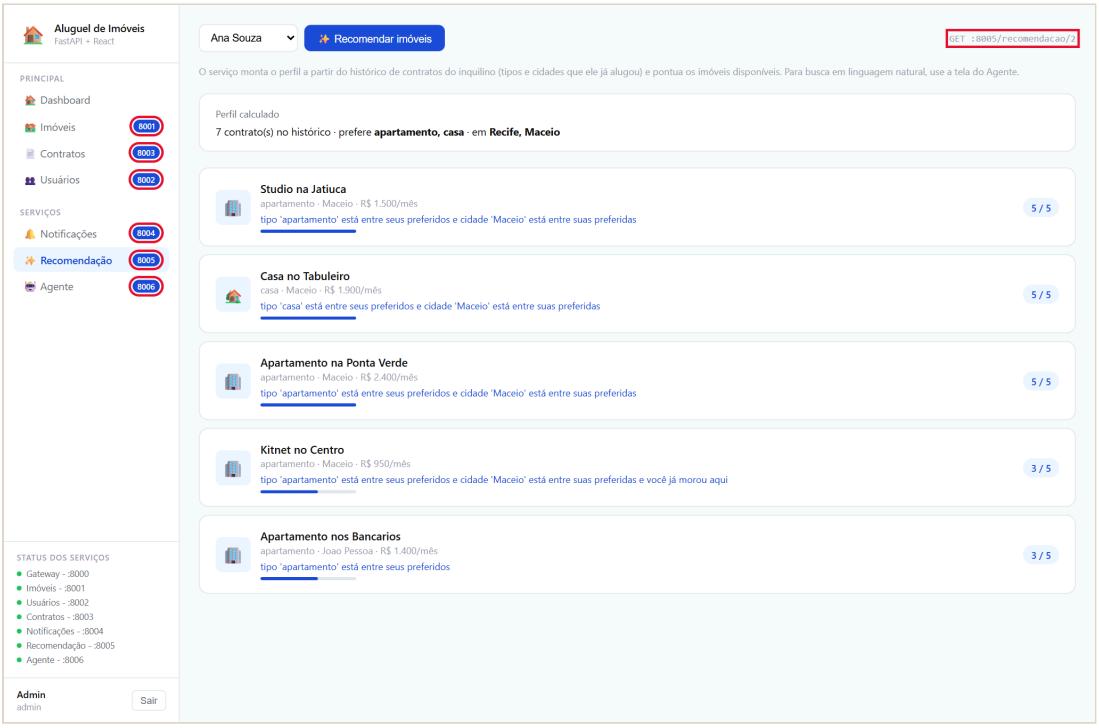


Figura 19 - Recomendação

Passo 13 - Agente

Busca em linguagem natural pelo serviço de Agente (:8006). Sem chave da API, o agente avisa o que falta e o resto do sistema segue funcionando.

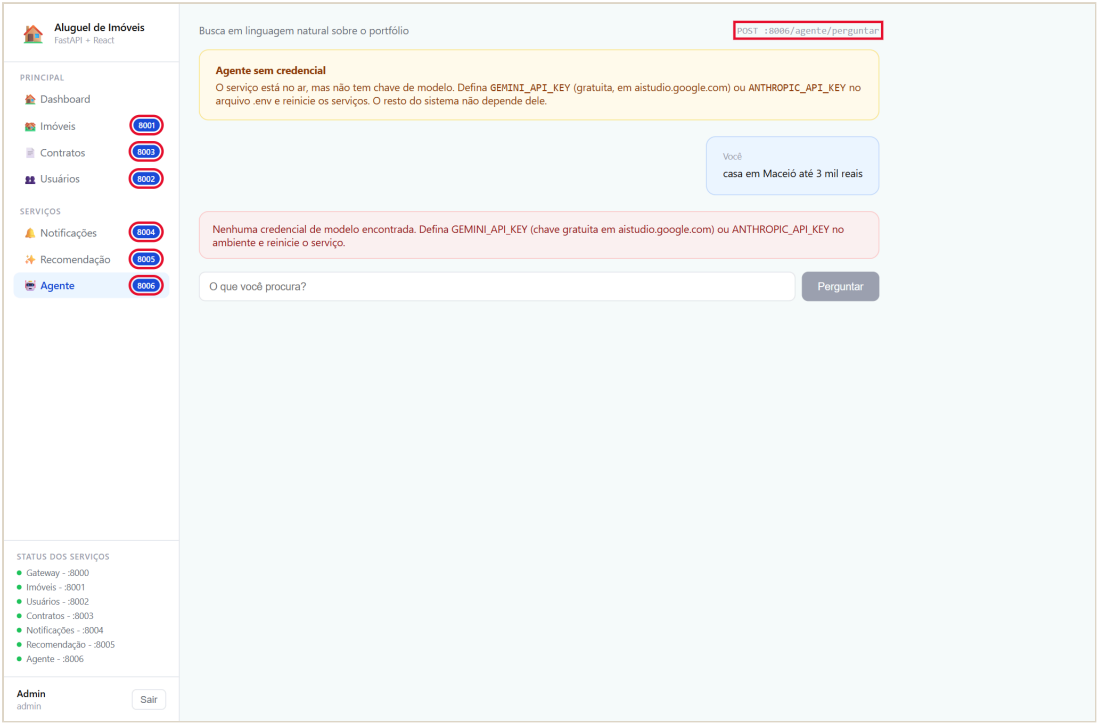


Figura 20 - Agente

Passo 14 - Visão da inquilina: catálogo

Logada como inquilina, a Ana consulta o catálogo, sem o botão de cadastrar imóvel. Pela API, a tentativa seria recusada com erro 403.

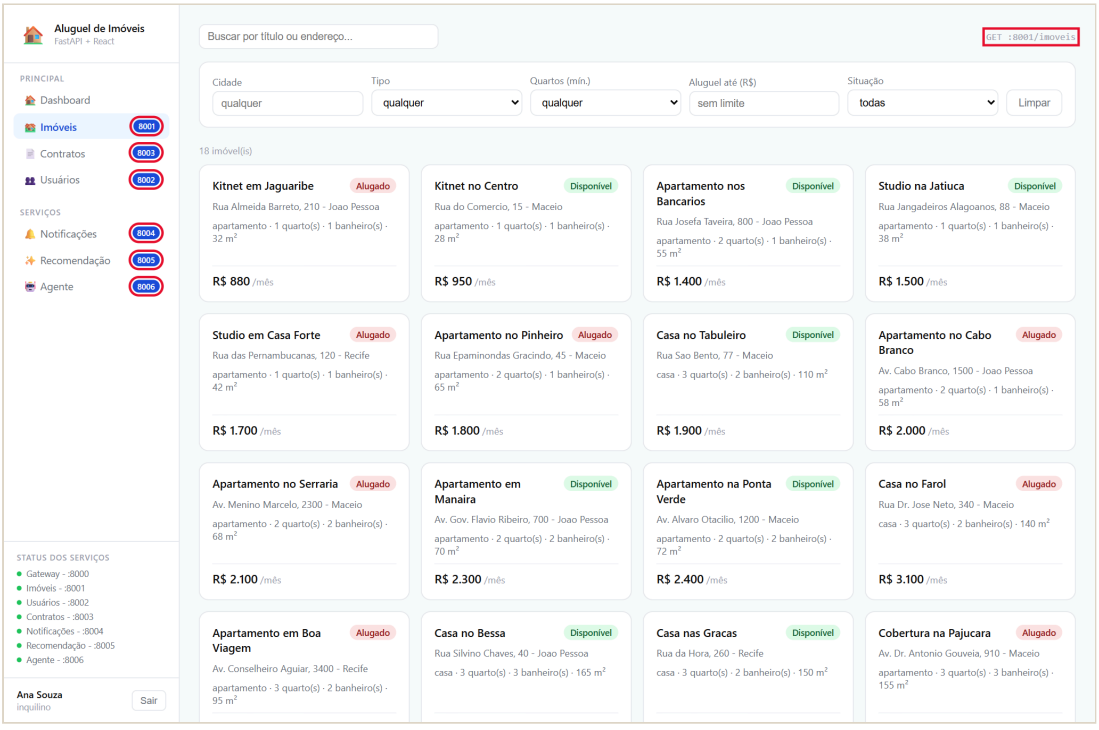


Figura 21 - Visão da inquilina: catálogo

Passo 15 - Visão da inquilina: contratos

A inquilina vê só os próprios contratos e só pode assinar em seu nome. A restrição é feita no serviço de Contratos com a identidade repassada pelo gateway.

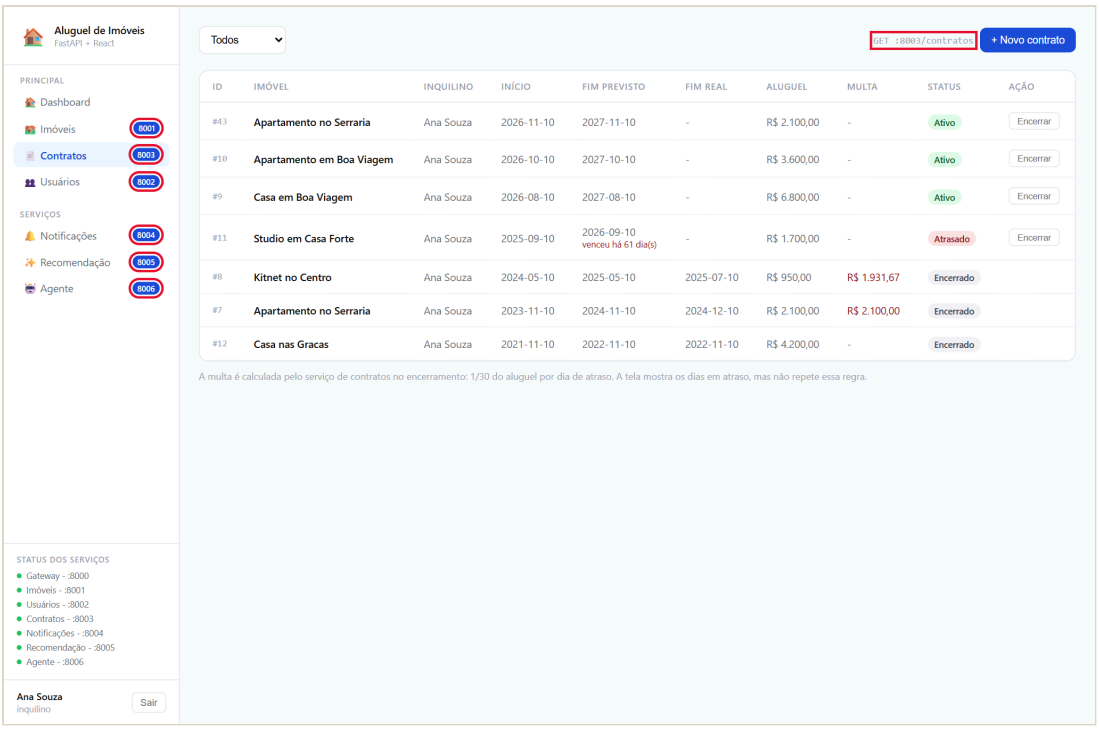


Figura 22 - Visão da inquilina: contratos

9 Código no GitHub e execução

Todo o código está em https://github.com/3runorocha/ECOM021-eng-de-software, com as instruções de execução no `README.md`. Estrutura:

```
ECOM021-eng-de-software/
├── biblioteca/
│   ├── gateway/            API Gateway (:8000)
│   ├── services/           imoveis, usuarios, contratos, notificacoes, recomendacao, agente
│   ├── framework/          interfaces, componentes e o framework (Template Method)
│   ├── apps/               aplicações derivadas do framework
│   └── seed.py, seed_contratos.py   dados de exemplo
├── biblioteca-online/      interface web (React + Vite, :5173)
├── tests/                  testes de unidade e de integração (pytest)
├── e2e/                    testes de ponta a ponta (Playwright)
├── diagramas/              diagramas em Mermaid
├── subir_servicos.py       sobe os sete processos
└── validar.py              validador do projeto
```

Como executar

1. Instale o Python 3 e o Node.js. Na raiz do projeto: `pip install -r biblioteca/requirements.txt`.
2. Suba os serviços: `python subir_servicos.py`. Confira em `http://localhost:8000/health`.
3. Em outro terminal, carregue os dados de exemplo: `cd biblioteca` e `python seed.py`. Para ter também contratos, pare os serviços, rode `python seed_contratos.py` e suba de novo.

4. Suba a interface: `cd biblioteca-online`, `npm install` e `npm run dev`; abra `http://localhost:5173`.
5. Entre com `admin@alugue.br` / `admin123` (admin), `beatriz@alugue.br` / `senha123` (proprietária) ou `ana@alugue.br` / `senha123` (inquilina).
6. Opcional: com `GEMINI_API_KEY` (gratuita, em aistudio.google.com) ou `ANTHROPIC_API_KEY` no arquivo `.env` da raiz, o agente passa a responder.

Para verificar: `python -m pytest` roda os testes de unidade; `python validar.py` sobe o sistema com bancos temporários e roda as 54 checagens; a pasta `e2e/` tem a suíte de ponta a ponta e um README próprio.

10 Referências bibliográficas

ANTHROPIC. *Claude API documentation: tool use*. Disponível em: https://docs.anthropic.com. Acesso em: 28 out. 2026.

GOOGLE. *Gemini API documentation: function calling*. Disponível em: https://aistudio.google.com/docs. Acesso em: 10 nov. 2026.

FASTAPI. *FastAPI documentation*. Disponível em: https://fastapi.tiangolo.com. Acesso em: 28 out. 2026.

FOWLER, M.; LEWIS, J. *Microservices: a definition of this new architectural term*. 2014. Disponível em: https://martinfowler.com/articles/microservices.html. Acesso em: 28 out. 2026.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. *Padrões de projeto: soluções reutilizáveis de software orientado a objetos*. Porto Alegre: Bookman, 2000.

NEWMAN, S. *Building microservices: designing fine-grained systems*. 2. ed. Sebastopol: O'Reilly Media, 2021.

PLAYWRIGHT. *Playwright for Python documentation*. Disponível em: https://playwright.dev/python. Acesso em: 28 out. 2026.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.

REACT. *React documentation*. Disponível em: https://react.dev. Acesso em: 28 out. 2026.

RICHARDSON, C. *Microservices patterns: with examples in Java*. Shelter Island: Manning, 2018.

SCHWABER, K.; SUTHERLAND, J. *The Scrum Guide*. 2020. Disponível em: https://scrumguides.org. Acesso em: 28 out. 2026.

SOMMERVILLE, I. *Engenharia de software*. 10. ed. São Paulo: Pearson, 2018.

SQLITE. *SQLite documentation*. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 28 out. 2026.

SZYPERSKI, C. *Component software: beyond object-oriented programming*. 2. ed. Boston: Addison-Wesley, 2002.

Apêndice A - Código documentado

Código dos arquivos que definem cada parte: rotas, regras de negócio e modelos de cada serviço, o gateway completo, os componentes, o framework e uma aplicação derivada. Arquivos de infraestrutura repetida (`main.py`, `repository.py` e `database.py` da maioria dos serviços) e a interface web estão no repositório. A documentação está nas docstrings e nos nomes; o projeto não usa comentários de linha.

A.1 API Gateway

biblioteca/gateway/config.py 33 linhas

```python
import os


SERVICES = {
    "imoveis":      {"url": os.getenv("IMOVEIS_URL",      "http://127.0.0.1:8001"), "porta": 8001, "descricao": "Portfolio de apartamentos e casas"},
    "usuarios":     {"url": os.getenv("USUARIOS_URL",     "http://127.0.0.1:8002"), "porta": 8002, "descricao": "Registro e autenticacao de usuarios"},
    "contratos":    {"url": os.getenv("CONTRATOS_URL",    "http://127.0.0.1:8003"), "porta": 8003, "descricao": "Locacao, encerramento e multa"},
    "notificacoes": {"url": os.getenv("NOTIFICACOES_URL", "http://127.0.0.1:8004"), "porta": 8004, "descricao": "Alertas de prazo e atraso"},
    "recomendacao": {"url": os.getenv("RECOMENDACAO_URL", "http://127.0.0.1:8005"), "porta": 8005, "descricao": "Recomendacao por perfil do usuario"},
    "agente":       {"url": os.getenv("AGENTE_URL",       "http://127.0.0.1:8006"), "porta": 8006, "descricao": "Busca conversacional sobre o portfolio"},
}

PUBLIC_ROUTES = {
    ("POST", "/usuarios/usuarios/registro"),
    ("POST", "/usuarios/usuarios/login"),
}

ROTAS_LIVRES = ["/health", "/services", "/docs", "/openapi.json", "/redoc"]


class ServiceRegistry:

    def __init__(self, services: dict):
        self._services = services

    def __contains__(self, nome: str) -> bool:
        return nome in self._services

    def url(self, nome: str) -> str:
        return self._services[nome]["url"]

    def itens(self):
        return self._services.items()
```

biblioteca/gateway/auth.py 70 linhas

```python
import httpx
from fastapi import Request, Response

from exceptions import GatewayError, TokenInvalido, ServicoIndisponivel


class AuthClient:

    def __init__(
        self, usuarios_url: str, client: httpx.AsyncClient, timeout: float = 5.0
    ):
        self._url = usuarios_url.rstrip("/")
        self._client = client
        self._timeout = timeout

    async def validar_token(self, token: str) -> dict:
        try:
            resp = await self._client.post(
                f"{self._url}/usuarios/validar-token",
                headers={"authorization": token},
```

```
                timeout=self._timeout,
            )
        except httpx.RequestError:
            raise ServicoIndisponivel("Servico de autenticacao indisponivel")
        if resp.status_code != 200:
            raise TokenInvalido("Token invalido ou expirado")
        return resp.json()


class Autenticador:

    def __init__(self, auth_client: AuthClient, public_routes: set, rotas_livres: list):
        self._auth = auth_client
        self._public_routes = public_routes
        self._rotas_livres = rotas_livres

    def _e_rota_livre(self, path: str) -> bool:
        return any(path.startswith(r) for r in self._rotas_livres)

    async def processar(self, request: Request, call_next):
        path = request.url.path
        method = request.method

        if self._e_rota_livre(path):
            return await call_next(request)
        if method == "OPTIONS":
            return await call_next(request)
        if (method, path) in self._public_routes:
            return await call_next(request)

        token = request.headers.get("authorization", "")
        if not token:
            return Response(
                content='{"detail": "Token de autenticacao obrigatorio"}',
                status_code=401,
                media_type="application/json",
            )

        try:
            identidade = await self._auth.validar_token(token)
        except GatewayError as e:
            return Response(
                content=f'{{"detail": "{e.detail}"}}',
                status_code=e.status_code,
                media_type="application/json",
            )

        request.state.identidade = identidade
        return await call_next(request)
```

biblioteca/gateway/proxy.py 125 linhas

```
import asyncio
from http.cookiejar import CookieJar, DefaultCookiePolicy

import httpx
from fastapi import Request, Response

from config import ServiceRegistry
from exceptions import ServicoNaoEncontrado, ServicoIndisponivel


def novo_cliente_http(transport=None) -> httpx.AsyncClient:
    """Cliente único do gateway, reaproveitando as conexões com os serviços.

    Por ser compartilhado entre todos os usuários, ele não guarda cookie: um
    Set-Cookie vindo de um serviço não pode ir parar na requisição de outra
    pessoa.
```

```python
    """
    sem_cookies = CookieJar(policy=DefaultCookiePolicy(allowed_domains=[]))
    return httpx.AsyncClient(cookies=sem_cookies, transport=transport)


class ProxyService:

    HEADERS_NAO_REPASSADOS = frozenset({
        "content-encoding",
        "content-length",
        "transfer-encoding",
        "connection",
    })

    HEADERS_DE_IDENTIDADE = frozenset({"x-usuario-id", "x-usuario-tipo"})

    def __init__(
        self, registry: ServiceRegistry, client: httpx.AsyncClient,
        timeout: float = 10.0,
    ):
        self._registry = registry
        self._client = client
        self._timeout = timeout

    async def encaminhar(
        self, request: Request, service_name: str, path: str
    ) -> Response:
        if service_name not in self._registry:
            raise ServicoNaoEncontrado(f"Servico '{service_name}' nao encontrado")

        target_url = f"{self._registry.url(service_name)}/{path}"
        params = dict(request.query_params)
        body = await request.body()
        headers = {
            chave: valor
            for chave, valor in request.headers.items()
            if chave.lower() != "host"
            and chave.lower() not in self.HEADERS_DE_IDENTIDADE
        }
        identidade = getattr(request.state, "identidade", None)
        if identidade:
            headers["x-usuario-id"] = str(identidade["usuario_id"])
            headers["x-usuario-tipo"] = identidade["tipo"]

        try:
            resp = await self._client.request(
                method=request.method,
                url=target_url,
                headers=headers,
                params=params,
                content=body,
                timeout=self._timeout,
            )
        except httpx.RequestError:
            raise ServicoIndisponivel(
                f"Microsservico '{service_name}' indisponivel"
            )

        headers_resposta = {
            chave: valor
            for chave, valor in resp.headers.items()
            if chave.lower() not in self.HEADERS_NAO_REPASSADOS
        }

        return Response(
            content=resp.content,
            status_code=resp.status_code,
            headers=headers_resposta,
            media_type=resp.headers.get("content-type"),
```

```
        )


class MonitorServicos:

    def __init__(
        self, registry: ServiceRegistry, client: httpx.AsyncClient,
        timeout: float = 3.0,
    ):
        self._registry = registry
        self._client = client
        self._timeout = timeout

    async def _consultar(self, info: dict) -> dict:
        try:
            resp = await self._client.get(
                f"{info['url']}/health", timeout=self._timeout
            )
            status = "ok" if resp.status_code == 200 else "erro"
        except httpx.RequestError:
            status = "indisponivel"
        return {"status": status, "porta": info["porta"]}

    async def verificar(self) -> dict:
        """Consulta todos os serviços ao mesmo tempo.

        Em série, com um serviço fora do ar o /health levava o timeout dele
        somado ao dos outros; em paralelo, leva o do mais lento.
        """
        itens = list(self._registry.itens())
        estados = await asyncio.gather(*(self._consultar(info) for _, info in itens))
        resultados = {nome: estado for (nome, _), estado in zip(itens, estados)}

        todos_ok = all(s["status"] == "ok" for s in resultados.values())
        return {
            "gateway": "ok",
            "status_geral": "ok" if todos_ok else "degradado",
            "servicos": resultados,
        }
```

biblioteca/gateway/main.py 97 linhas

```
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from fastapi.middleware.cors import CORSMiddleware

from config import SERVICES, PUBLIC_ROUTES, ROTAS_LIVRES, ServiceRegistry
from auth import AuthClient, Autenticador
from proxy import ProxyService, MonitorServicos, novo_cliente_http
from exceptions import GatewayError

registry = ServiceRegistry(SERVICES)
cliente_http = novo_cliente_http()
auth_client = AuthClient(registry.url("usuarios"), cliente_http)
autenticador = Autenticador(auth_client, PUBLIC_ROUTES, ROTAS_LIVRES)
proxy_service = ProxyService(registry, cliente_http)
monitor = MonitorServicos(registry, cliente_http)


@asynccontextmanager
async def lifespan(app: FastAPI):
    yield
    await cliente_http.aclose()


app = FastAPI(
```

```
    lifespan=lifespan,
    title="Aluguel de Imoveis - API Gateway",
    description="Ponto de entrada unico para todos os microsservicos",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)


@app.exception_handler(GatewayError)
async def tratar_gateway_error(request: Request, exc: GatewayError):
    return JSONResponse(status_code=exc.status_code, content={"detail": exc.detail})


@app.middleware("http")
async def middleware_autenticacao(request: Request, call_next):
    return await autenticador.processar(request, call_next)


METODOS = ["GET", "POST", "PATCH", "DELETE", "PUT"]


@app.api_route("/imoveis/{path:path}", methods=METODOS)
async def gateway_imoveis(request: Request, path: str):
    return await proxy_service.encaminhar(request, "imoveis", path)


@app.api_route("/usuarios/{path:path}", methods=METODOS)
async def gateway_usuarios(request: Request, path: str):
    return await proxy_service.encaminhar(request, "usuarios", path)


@app.api_route("/contratos/{path:path}", methods=METODOS)
async def gateway_contratos(request: Request, path: str):
    return await proxy_service.encaminhar(request, "contratos", path)


@app.api_route("/notificacoes/{path:path}", methods=METODOS)
async def gateway_notificacoes(request: Request, path: str):
    return await proxy_service.encaminhar(request, "notificacoes", path)


@app.api_route("/recomendacao/{path:path}", methods=METODOS)
async def gateway_recomendacao(request: Request, path: str):
    return await proxy_service.encaminhar(request, "recomendacao", path)


@app.api_route("/agente/{path:path}", methods=METODOS)
async def gateway_agente(request: Request, path: str):
    return await proxy_service.encaminhar(request, "agente", path)


@app.get("/health")
async def health_check():
    return await monitor.verificar()


@app.get("/services")
def listar_servicos():
    return {
        "gateway": {"porta": 8000, "descricao": "API Gateway - ponto de entrada unico"},
        **{
            nome: {"porta": info["porta"], "descricao": info["descricao"]}
            for nome, info in registry.itens()
```

```
        },
    }
```

biblioteca/gateway/exceptions.py 18 linhas

```python
class GatewayError(Exception):
    status_code = 500

    def __init__(self, detail: str):
        self.detail = detail
        super().__init__(detail)


class ServicoNaoEncontrado(GatewayError):
    status_code = 404


class TokenInvalido(GatewayError):
    status_code = 401


class ServicoIndisponivel(GatewayError):
    status_code = 502
```

A.2 Microsserviço de Imóveis

biblioteca/services/imoveis/models.py 40 linhas

```python
from typing import Literal, Optional
from pydantic import BaseModel, Field

TipoImovel = Literal["apartamento", "casa"]


class ImovelCreate(BaseModel):
    titulo: str
    tipo: TipoImovel
    endereco: str
    cidade: str
    quartos: int = Field(ge=0)
    banheiros: int = Field(ge=0)
    area_m2: float = Field(gt=0)
    valor_mensal: float = Field(gt=0)


class ImovelUpdate(BaseModel):
    titulo: Optional[str] = None
    tipo: Optional[TipoImovel] = None
    endereco: Optional[str] = None
    cidade: Optional[str] = None
    quartos: Optional[int] = Field(default=None, ge=0)
    banheiros: Optional[int] = Field(default=None, ge=0)
    area_m2: Optional[float] = Field(default=None, gt=0)
    valor_mensal: Optional[float] = Field(default=None, gt=0)


class Imovel(BaseModel):
    id: int
    titulo: str
    tipo: TipoImovel
    endereco: str
    cidade: str
    quartos: int
    banheiros: int
    area_m2: float
    valor_mensal: float
    disponivel: bool
```

biblioteca/services/imoveis/routes.py 60 linhas

```python
from typing import Optional

from fastapi import APIRouter, Depends, Header

from models import Imovel, ImovelCreate, ImovelUpdate
from service import ImovelService


def identificar(
    x_usuario_id: Optional[int] = Header(None),
    x_usuario_tipo: Optional[str] = Header(None),
) -> Optional[dict]:
    if x_usuario_id is None:
        return None
    return {"id": x_usuario_id, "tipo": x_usuario_tipo or ""}


def criar_router(service: ImovelService) -> APIRouter:
    router = APIRouter(prefix="/imoveis", tags=["Imóveis"])

```

```python
    @router.get("/", response_model=list[Imovel])
    def listar_imoveis(
        cidade: str = None,
        tipo: str = None,
        quartos_min: int = None,
        valor_min: float = None,
        valor_max: float = None,
        disponivel: bool = None,
    ):
        return service.listar_imoveis(
            cidade=cidade, tipo=tipo, quartos_min=quartos_min,
            valor_min=valor_min, valor_max=valor_max, disponivel=disponivel,
        )

    @router.get("/{imovel_id}", response_model=Imovel)
    def buscar_imovel(imovel_id: int):
        return service.buscar_imovel(imovel_id)

    @router.post("/", response_model=Imovel, status_code=201)
    def cadastrar_imovel(imovel: ImovelCreate, chamador=Depends(identificar)):
        return service.cadastrar_imovel(imovel, chamador)

    @router.patch("/{imovel_id}", response_model=Imovel)
    def atualizar_imovel(
        imovel_id: int, dados: ImovelUpdate, chamador=Depends(identificar)
    ):
        return service.atualizar_imovel(imovel_id, dados, chamador)

    @router.delete("/{imovel_id}", status_code=204)
    def remover_imovel(imovel_id: int, chamador=Depends(identificar)):
        service.remover_imovel(imovel_id, chamador)

    @router.patch("/{imovel_id}/disponibilidade")
    def definir_disponibilidade(
        imovel_id: int, disponivel: bool, chamador=Depends(identificar)
    ):
        return service.definir_disponibilidade(imovel_id, disponivel, chamador)

    return router
```

biblioteca/services/imoveis/service.py 94 linhas

```python
from models import Imovel, ImovelCreate, ImovelUpdate
from repository import ImovelRepository
from exceptions import ImovelNaoEncontrado, PermissaoNegada, RegraDeNegocio

PODEM_ADMINISTRAR_PORTFOLIO = {"admin", "proprietario"}


def exigir(chamador: dict | None, permitidos: set, acao: str) -> None:
    if chamador is not None and chamador["tipo"] not in permitidos:
        raise PermissaoNegada(f"Seu perfil não pode {acao}")


class ImovelService:

    def __init__(self, repository: ImovelRepository):
        self._repo = repository

    def listar_imoveis(
        self,
        cidade: str = None,
        tipo: str = None,
        quartos_min: int = None,
        valor_min: float = None,
        valor_max: float = None,
        disponivel: bool = None,
    ) -> list[Imovel]:
```

```python
        if valor_min is not None and valor_max is not None and valor_min > valor_max:
            raise RegraDeNegocio("valor_min não pode ser maior que valor_max")
        return self._repo.listar(
            cidade=cidade, tipo=tipo, quartos_min=quartos_min,
            valor_min=valor_min, valor_max=valor_max, disponivel=disponivel,
        )

    def buscar_imovel(self, imovel_id: int) -> Imovel:
        imovel = self._repo.buscar_por_id(imovel_id)
        if imovel is None:
            raise ImovelNaoEncontrado()
        return imovel

    def cadastrar_imovel(
        self, dados: ImovelCreate, chamador: dict | None = None
    ) -> Imovel:
        exigir(chamador, PODEM_ADMINISTRAR_PORTFOLIO, "cadastrar imóvel")
        novo_id = self._repo.inserir(dados)
        return self._repo.buscar_por_id(novo_id)

    def atualizar_imovel(
        self, imovel_id: int, dados: ImovelUpdate, chamador: dict | None = None
    ) -> Imovel:
        exigir(chamador, PODEM_ADMINISTRAR_PORTFOLIO, "alterar imóvel")
        atual = self._repo.buscar_por_id(imovel_id)
        if atual is None:
            raise ImovelNaoEncontrado()

        campos = dados.model_dump(exclude_none=True)
        if not campos:
            return atual

        self._repo.atualizar(imovel_id, campos)
        return self._repo.buscar_por_id(imovel_id)

    def remover_imovel(self, imovel_id: int, chamador: dict | None = None) -> None:
        exigir(chamador, PODEM_ADMINISTRAR_PORTFOLIO, "remover imóvel")
        atual = self._repo.buscar_por_id(imovel_id)
        if atual is None:
            raise ImovelNaoEncontrado()
        if not atual.disponivel:
            raise RegraDeNegocio(
                f"Imóvel '{atual.titulo}' está alugado; encerre o contrato antes de remover"
            )
        self._repo.remover(imovel_id)

    def definir_disponibilidade(
        self, imovel_id: int, disponivel: bool, chamador: dict | None = None
    ) -> dict:
        """Substitui o antigo atualizar_disponibilidade(delta).

        Um imovel nao tem N exemplares: ou esta livre, ou esta alugado. Exigir
        que o estado mude de fato preserva a guarda que o acervo tinha ("nenhum
        exemplar disponivel") e da ao servico de contratos um 400 para tratar
        quando duas locacoes disputam o mesmo imovel.
        """
        exigir(chamador, {"admin"}, "mudar a disponibilidade à mão")
        imovel = self._repo.buscar_por_id(imovel_id)
        if imovel is None:
            raise ImovelNaoEncontrado()

        if imovel.disponivel == disponivel:
            estado = "disponível" if disponivel else "indisponível"
            raise RegraDeNegocio(f"Imóvel '{imovel.titulo}' já está {estado}")

        self._repo.atualizar(imovel_id, {"disponivel": int(disponivel)})
        return {"imovel_id": imovel_id, "disponivel": disponivel}
```

biblioteca/services/imoveis/database.py 45 linhas

```python
import sqlite3
import unicodedata


def sem_acento(texto):
    if texto is None:
        return None
    decomposto = unicodedata.normalize("NFKD", texto)
    return "".join(c for c in decomposto if not unicodedata.combining(c)).casefold()


class Database:

    def __init__(self, db_path: str):
        self._db_path = db_path

    def connect(self) -> sqlite3.Connection:
        conn = sqlite3.connect(self._db_path)
        conn.row_factory = sqlite3.Row
        conn.create_function("sem_acento", 1, sem_acento, deterministic=True)
        return conn

    def init_schema(self) -> None:
        conn = self.connect()
        try:
            conn.execute(
                """
                CREATE TABLE IF NOT EXISTS imoveis (
                    id           INTEGER PRIMARY KEY AUTOINCREMENT,
                    titulo       TEXT    NOT NULL,
                    tipo         TEXT    NOT NULL,
                    endereco     TEXT    NOT NULL,
                    cidade       TEXT    NOT NULL,
                    quartos      INTEGER NOT NULL,
                    banheiros    INTEGER NOT NULL,
                    area_m2      REAL    NOT NULL,
                    valor_mensal REAL    NOT NULL,
                    disponivel   INTEGER NOT NULL DEFAULT 1
                )
                """
            )
            conn.commit()
        finally:
            conn.close()
```

A.3 Microsserviço de Usuários

biblioteca/services/usuarios/models.py 32 linhas

```python
from typing import Literal, Optional
from pydantic import BaseModel

TipoUsuario = Literal["inquilino", "proprietario", "admin"]


class UsuarioCreate(BaseModel):
    nome: str
    email: str
    senha: str
    tipo: TipoUsuario = "inquilino"


class UsuarioLogin(BaseModel):
    email: str
    senha: str


class UsuarioPublico(BaseModel):
    id: int
    nome: str
    email: str
    tipo: str
    ativo: bool


class UsuarioUpdate(BaseModel):
    nome: Optional[str] = None
    email: Optional[str] = None
    tipo: Optional[TipoUsuario] = None
    ativo: Optional[bool] = None
```

biblioteca/services/usuarios/routes.py 36 linhas

```python
from fastapi import APIRouter, Header
from models import UsuarioCreate, UsuarioLogin, UsuarioPublico, UsuarioUpdate
from service import UsuarioService


def criar_router(service: UsuarioService) -> APIRouter:
    router = APIRouter(prefix="/usuarios", tags=["Usuários"])

    @router.post("/registro", response_model=UsuarioPublico, status_code=201)
    def registrar(usuario: UsuarioCreate):
        return service.registrar(usuario)

    @router.post("/login")
    def login(credenciais: UsuarioLogin):
        return service.login(credenciais)

    @router.get("/", response_model=list[UsuarioPublico])
    def listar_usuarios():
        return service.listar_usuarios()

    @router.get("/{usuario_id}", response_model=UsuarioPublico)
    def buscar_usuario(usuario_id: int):
        return service.buscar_usuario(usuario_id)

    @router.patch("/{usuario_id}", response_model=UsuarioPublico)
    def atualizar_usuario(
        usuario_id: int, dados: UsuarioUpdate, authorization: str = Header(...)
    ):
```

```
        return service.atualizar_usuario(authorization, usuario_id, dados)

    @router.post("/validar-token")
    def validar(authorization: str = Header(...)):
        return service.validar_token(authorization)

    return router
```

biblioteca/services/usuarios/service.py 95 linhas

```python
from models import UsuarioCreate, UsuarioLogin, UsuarioPublico, UsuarioUpdate
from repository import UsuarioRepository
from security import PasswordHasher, TokenService
from exceptions import (
    UsuarioNaoEncontrado, CredenciaisInvalidas, PermissaoNegada, TokenInvalido,
)

CAMPOS_SO_DE_ADMIN = {"tipo", "ativo"}


class UsuarioService:

    def __init__(
        self,
        repository: UsuarioRepository,
        hasher: PasswordHasher,
        token_service: TokenService,
    ):
        self._repo = repository
        self._hasher = hasher
        self._tokens = token_service
        self._hash_falso = hasher.hash("senha-para-equalizar-tempo")

    def registrar(self, usuario: UsuarioCreate) -> UsuarioPublico:
        if usuario.tipo != "inquilino" and self._repo.existe_algum():
            raise PermissaoNegada(
                "Cadastro público cria só inquilino; proprietário e admin são "
                "definidos por um admin (só o primeiro usuário nasce admin)"
            )
        senha_hash = self._hasher.hash(usuario.senha)
        novo_id = self._repo.inserir(
            usuario.nome, usuario.email, senha_hash, usuario.tipo
        )
        return self._repo.buscar_por_id(novo_id)

    def login(self, credenciais: UsuarioLogin) -> dict:
        encontrado = self._repo.buscar_ativo_com_hash(credenciais.email)
        if encontrado is None:
            self._hasher.verificar(credenciais.senha, self._hash_falso)
            raise CredenciaisInvalidas()

        usuario, senha_hash = encontrado
        if not self._hasher.verificar(credenciais.senha, senha_hash):
            raise CredenciaisInvalidas()

        token = self._tokens.gerar(usuario.id, usuario.email)
        return {
            "token": token,
            "usuario": {
                "id": usuario.id,
                "nome": usuario.nome,
                "email": usuario.email,
                "tipo": usuario.tipo,
            },
        }

    def listar_usuarios(self) -> list[UsuarioPublico]:
        return self._repo.listar()
```

```
59
60      def buscar_usuario(self, usuario_id: int) -> UsuarioPublico:
61          usuario = self._repo.buscar_por_id(usuario_id)
62          if usuario is None:
63              raise UsuarioNaoEncontrado()
64          return usuario
65
66      def atualizar_usuario(
67          self, authorization: str, usuario_id: int, dados: UsuarioUpdate
68      ) -> UsuarioPublico:
69          chamador = self.buscar_usuario(self.validar_token(authorization)["usuario_id"])
70
71          atual = self._repo.buscar_por_id(usuario_id)
72          if atual is None:
73              raise UsuarioNaoEncontrado()
74
75          e_admin = chamador.tipo == "admin"
76          if chamador.id != usuario_id and not e_admin:
77              raise PermissaoNegada("Só o próprio usuário ou um admin altera este cadastro")
78
79          campos = dados.model_dump(exclude_none=True)
80          if CAMPOS_SO_DE_ADMIN & campos.keys() and not e_admin:
81              raise PermissaoNegada("Só admin altera tipo ou situação de um usuário")
82          if not campos:
83              return atual
84
85          self._repo.atualizar(usuario_id, campos)
86          return self._repo.buscar_por_id(usuario_id)
87
88      def validar_token(self, authorization: str) -> dict:
89          token = authorization.replace("Bearer ", "")
90          payload = self._tokens.validar(token)
91          usuario = self._repo.buscar_por_id(payload["usuario_id"])
92          if usuario is None or not usuario.ativo:
93              raise TokenInvalido()
94          return {"valido": True, **payload, "tipo": usuario.tipo}
```

biblioteca/services/usuarios/security.py 70 linhas

```
1  import hashlib
2  import hmac
3  import secrets
4  from datetime import datetime, timedelta, timezone
5
6  from exceptions import TokenInvalido
7
8
9  class PasswordHasher:
10
11      ALGORITMO = "pbkdf2_sha256"
12      ITERACOES = 600_000
13
14      def hash(self, senha: str) -> str:
15          sal = secrets.token_hex(16)
16          derivada = self._derivar(senha, sal, self.ITERACOES)
17          return f"{self.ALGORITMO}${self.ITERACOES}${sal}${derivada}"
18
19      def verificar(self, senha: str, guardado: str) -> bool:
20          try:
21              algoritmo, iteracoes, sal, esperada = guardado.split("$")
22              iteracoes = int(iteracoes)
23          except (ValueError, AttributeError):
24              return False
25          if algoritmo != self.ALGORITMO:
26              return False
27          return hmac.compare_digest(self._derivar(senha, sal, iteracoes), esperada)
28
29      @staticmethod
```

```
    def _derivar(senha: str, sal: str, iteracoes: int) -> str:
        return hashlib.pbkdf2_hmac(
            "sha256", senha.encode(), bytes.fromhex(sal), iteracoes
        ).hex()


class TokenService:

    def __init__(self, secret: str, horas_validade: int = 8):
        if not secret:
            raise ValueError("TokenService exige um segredo não vazio")
        self._secret = secret.encode()
        self._horas_validade = horas_validade

    @staticmethod
    def _agora() -> datetime:
        return datetime.now(timezone.utc)

    def _assinar(self, raw: str) -> str:
        return hmac.new(self._secret, raw.encode(), hashlib.sha256).hexdigest()

    def gerar(self, usuario_id: int, email: str) -> str:
        nonce = secrets.token_hex(16)
        expires = int(
            (self._agora() + timedelta(hours=self._horas_validade)).timestamp()
        )
        raw = f"{usuario_id}:{email}:{expires}:{nonce}"
        return f"{raw}:{self._assinar(raw)}"

    def validar(self, token: str) -> dict:
        try:
            usuario_id, email, expires, nonce, signature = token.split(":")
            raw = f"{usuario_id}:{email}:{expires}:{nonce}"
            if not hmac.compare_digest(signature, self._assinar(raw)):
                raise ValueError("Assinatura inválida")
            if int(expires) < int(self._agora().timestamp()):
                raise ValueError("Token expirado")
            return {"usuario_id": int(usuario_id), "email": email}
        except Exception:
            raise TokenInvalido()
```

A.4 Microsserviço de Contratos

biblioteca/services/contratos/models.py 32 linhas

```python
from typing import Optional
from pydantic import BaseModel, Field

STATUS_EM_ABERTO = ("ativo", "atrasado")


class Chamador(BaseModel):
    id: int
    tipo: str

    @property
    def admin(self) -> bool:
        return self.tipo == "admin"


class ContratoCreate(BaseModel):
    inquilino_id: int
    imovel_id: int
    meses: Optional[int] = Field(default=None, ge=1, le=120)


class Contrato(BaseModel):
    id: int
    inquilino_id: int
    imovel_id: int
    data_inicio: str
    data_fim_prevista: str
    data_fim_real: Optional[str]
    valor_mensal: float
    status: str
    multa: float
```

biblioteca/services/contratos/routes.py 40 linhas

```python
from typing import Optional

from fastapi import APIRouter, Depends, Header

from models import Chamador, Contrato, ContratoCreate
from service import ContratoService


def identificar(
    x_usuario_id: Optional[int] = Header(None),
    x_usuario_tipo: Optional[str] = Header(None),
) -> Optional[Chamador]:
    if x_usuario_id is None:
        return None
    return Chamador(id=x_usuario_id, tipo=x_usuario_tipo or "")


def criar_router(service: ContratoService) -> APIRouter:
    router = APIRouter(prefix="/contratos", tags=["Contratos"])

    @router.post("/", response_model=Contrato, status_code=201)
    def registrar_contrato(dados: ContratoCreate, chamador=Depends(identificar)):
        return service.registrar_contrato(dados, chamador)

    @router.post("/{contrato_id}/encerrar", response_model=Contrato)
    def encerrar_contrato(contrato_id: int, chamador=Depends(identificar)):
        return service.encerrar_contrato(contrato_id, chamador)

```

```
    @router.get("/", response_model=list[Contrato])
    def listar_contratos(
        inquilino_id: int = None, status: str = None, chamador=Depends(identificar)
    ):
        return service.listar_contratos(inquilino_id, status, chamador)

    @router.get("/{contrato_id}", response_model=Contrato)
    def buscar_contrato(contrato_id: int, chamador=Depends(identificar)):
        return service.buscar_contrato(contrato_id, chamador)

    return router
```

biblioteca/services/contratos/service.py 134 linhas

```python
import calendar
from datetime import date

from models import Chamador, Contrato, ContratoCreate
from repository import ContratoRepository
from clients import ImovelClient
from exceptions import ContratoNaoEncontrado, PermissaoNegada, RegraDeNegocio


def somar_meses(inicio: date, meses: int) -> date:
    """Soma meses preservando o dia, ajustando quando o mes de destino e curto.

    31/01 + 1 mes = 28/02 (ou 29 em ano bissexto). timedelta nao resolve isso
    porque mes nao tem duracao fixa.
    """
    total = inicio.month - 1 + meses
    ano = inicio.year + total // 12
    mes = total % 12 + 1
    dia = min(inicio.day, calendar.monthrange(ano, mes)[1])
    return date(ano, mes, dia)


class ContratoService:

    PRAZO_MESES = 12
    DIAS_BASE_MULTA = 30

    def __init__(self, repository: ContratoRepository, imoveis: ImovelClient):
        self._repo = repository
        self._imoveis = imoveis

    @staticmethod
    def _restrito(chamador: Chamador | None) -> bool:
        return chamador is not None and not chamador.admin

    def _visivel(self, contrato_id: int, chamador: Chamador | None) -> Contrato:
        contrato = self._repo.buscar_por_id(contrato_id)
        if contrato is None:
            raise ContratoNaoEncontrado()
        if self._restrito(chamador) and contrato.inquilino_id != chamador.id:
            raise ContratoNaoEncontrado()
        return contrato

    def registrar_contrato(
        self, dados: ContratoCreate, chamador: Chamador | None = None
    ) -> Contrato:
        if self._restrito(chamador) and dados.inquilino_id != chamador.id:
            raise PermissaoNegada("Só o admin registra contrato em nome de outro inquilino")

        imovel = self._imoveis.buscar_imovel(dados.imovel_id)
        if imovel is None:
            raise RegraDeNegocio(f"Imóvel {dados.imovel_id} não encontrado")

        if self._repo.existe_aberto(dados.imovel_id):
```

```
            raise RegraDeNegocio(
                f"Imóvel '{imovel['titulo']}' já possui contrato em aberto"
            )

        meses = dados.meses or self.PRAZO_MESES
        inicio = date.today()
        fim_previsto = somar_meses(inicio, meses)

        self._imoveis.definir_disponibilidade(dados.imovel_id, False)
        try:
            novo_id = self._repo.inserir(
                dados.inquilino_id,
                dados.imovel_id,
                str(inicio),
                str(fim_previsto),
                imovel["valor_mensal"],
            )
        except Exception:
            self._imoveis.definir_disponibilidade(dados.imovel_id, True)
            raise

        return self._repo.buscar_por_id(novo_id)

    def encerrar_contrato(
        self, contrato_id: int, chamador: Chamador | None = None
    ) -> Contrato:
        contrato = self._visivel(contrato_id, chamador)
        if contrato.status == "encerrado":
            raise RegraDeNegocio("Contrato já foi encerrado")

        hoje = date.today()
        multa = self.calcular_multa(contrato, hoje)

        liberou = self._liberar(contrato.imovel_id)
        try:
            self._repo.encerrar(contrato_id, str(hoje), multa)
        except Exception:
            if liberou:
                self._imoveis.definir_disponibilidade(contrato.imovel_id, False)
            raise
        return self._repo.buscar_por_id(contrato_id)

    def _liberar(self, imovel_id: int) -> bool:
        """Libera o imóvel; devolve False se ele já estava livre ou não existe mais.

        Nenhum dos dois casos pode impedir o encerramento: o contrato acabou
        de qualquer forma, e o imóvel já está no estado que se queria.
        """
        try:
            self._imoveis.definir_disponibilidade(imovel_id, True)
        except RegraDeNegocio:
            return False
        return True

    def calcular_multa(self, contrato: Contrato, data_saida: date) -> float:
        prevista = date.fromisoformat(contrato.data_fim_prevista)
        dias_atraso = max(0, (data_saida - prevista).days)
        if dias_atraso == 0:
            return 0.0
        valor_dia = contrato.valor_mensal / self.DIAS_BASE_MULTA
        return round(dias_atraso * valor_dia, 2)

    def listar_contratos(
        self,
        inquilino_id: int = None,
        status: str = None,
        chamador: Chamador | None = None,
    ) -> list[Contrato]:
        if self._restrito(chamador):
```

```
            if inquilino_id not in (None, chamador.id):
                raise PermissaoNegada("Só o admin lista contratos de outro inquilino")
            inquilino_id = chamador.id
        self._repo.marcar_atrasados(str(date.today()))
        return self._repo.listar(inquilino_id, status)

    def buscar_contrato(
        self, contrato_id: int, chamador: Chamador | None = None
    ) -> Contrato:
        return self._visivel(contrato_id, chamador)
```

biblioteca/services/contratos/clients.py 36 linhas

```python
import httpx

from exceptions import RegraDeNegocio, ServicoIndisponivel


class ImovelClient:

    def __init__(self, base_url: str, timeout: float = 5.0):
        self._base_url = base_url.rstrip("/")
        self._timeout = timeout

    def buscar_imovel(self, imovel_id: int) -> dict | None:
        try:
            resp = httpx.get(
                f"{self._base_url}/imoveis/{imovel_id}", timeout=self._timeout
            )
            if resp.status_code == 404:
                return None
            resp.raise_for_status()
            return resp.json()
        except httpx.RequestError:
            raise ServicoIndisponivel("Serviço de Imóveis indisponível")

    def definir_disponibilidade(self, imovel_id: int, disponivel: bool) -> None:
        try:
            resp = httpx.patch(
                f"{self._base_url}/imoveis/{imovel_id}/disponibilidade",
                params={"disponivel": disponivel},
                timeout=self._timeout,
            )
            if resp.status_code in (400, 404):
                raise RegraDeNegocio(resp.json().get("detail"))
            resp.raise_for_status()
        except httpx.RequestError:
            raise ServicoIndisponivel("Serviço de Imóveis indisponível")
```

A.5 Microsserviço de Notificações

biblioteca/services/notificacoes/models.py 19 linhas

```python
from typing import Optional
from pydantic import BaseModel


class NotificacaoCreate(BaseModel):
    usuario_id: int
    tipo: str
    mensagem: str
    imovel_id: Optional[int] = None


class Notificacao(BaseModel):
    id: int
    usuario_id: int
    tipo: str
    mensagem: str
    imovel_id: Optional[int]
    lida: bool
    data_criacao: str
```

biblioteca/services/notificacoes/routes.py 30 linhas

```python
from fastapi import APIRouter

from models import Notificacao, NotificacaoCreate
from service import NotificacaoService


def criar_router(service: NotificacaoService) -> APIRouter:
    router = APIRouter(prefix="/notificacoes", tags=["Notificações"])

    @router.post("/", response_model=Notificacao, status_code=201)
    def criar_notificacao(dados: NotificacaoCreate):
        return service.criar_notificacao(dados)

    @router.get("/usuario/{usuario_id}", response_model=list[Notificacao])
    def listar_por_usuario(usuario_id: int, apenas_nao_lidas: bool = False):
        return service.listar_por_usuario(usuario_id, apenas_nao_lidas)

    @router.patch("/{notificacao_id}/ler", response_model=Notificacao)
    def marcar_como_lida(notificacao_id: int):
        return service.marcar_como_lida(notificacao_id)

    @router.patch("/usuario/{usuario_id}/ler-todas")
    def marcar_todas_como_lidas(usuario_id: int):
        return service.marcar_todas_como_lidas(usuario_id)

    @router.post("/varredura")
    def varredura_automatica():
        return service.executar_varredura()

    return router
```

biblioteca/services/notificacoes/service.py 106 linhas

```python
from datetime import date

from models import Notificacao, NotificacaoCreate
from repository import NotificacaoRepository
from clients import ContratoClient
from exceptions import NotificacaoNaoEncontrada
```

```python


class NotificacaoService:

    DIAS_ALERTA_PRAZO = 2
    DIAS_BASE_MULTA = 30
    TIPOS_AUTOMATICOS = ("atraso", "prazo_proximo")

    def __init__(
        self, repository: NotificacaoRepository, contratos: ContratoClient
    ):
        self._repo = repository
        self._contratos = contratos

    def criar_notificacao(self, dados: NotificacaoCreate) -> Notificacao:
        novo_id = self._repo.inserir(dados)
        return self._repo.buscar_por_id(novo_id)

    def listar_por_usuario(
        self, usuario_id: int, apenas_nao_lidas: bool = False
    ) -> list[Notificacao]:
        return self._repo.listar_por_usuario(usuario_id, apenas_nao_lidas)

    def marcar_como_lida(self, notificacao_id: int) -> Notificacao:
        notificacao = self._repo.buscar_por_id(notificacao_id)
        if notificacao is None:
            raise NotificacaoNaoEncontrada()
        self._repo.marcar_como_lida(notificacao_id)
        return self._repo.buscar_por_id(notificacao_id)

    def marcar_todas_como_lidas(self, usuario_id: int) -> dict:
        self._repo.marcar_todas_como_lidas(usuario_id)
        return {
            "mensagem": (
                f"Todas as notificações do usuário {usuario_id} "
                "marcadas como lidas"
            )
        }

    def executar_varredura(self) -> dict:
        contratos = self._contratos.listar_contratos()
        hoje = date.today()
        geradas = 0

        for contrato in contratos:
            if contrato["status"] == "encerrado":
                continue

            inquilino_id = contrato["inquilino_id"]
            imovel_id = contrato["imovel_id"]
            prevista = date.fromisoformat(contrato["data_fim_prevista"])
            dias_restantes = (prevista - hoje).days

            if self._repo.existe_nao_lida(
                inquilino_id, imovel_id, self.TIPOS_AUTOMATICOS
            ):
                continue

            notificacao = self._montar_notificacao(
                contrato, inquilino_id, imovel_id, dias_restantes
            )
            if notificacao is not None:
                self._repo.inserir(notificacao)
                geradas += 1

        return {
            "mensagem": f"Varredura concluída. {geradas} notificação(ões) gerada(s)."
        }

```

```python
    def _montar_notificacao(
        self, contrato: dict, inquilino_id: int, imovel_id: int, dias_restantes: int
    ) -> NotificacaoCreate | None:
        if contrato["status"] == "atrasado":
            dias_atraso = abs(dias_restantes)
            valor_dia = contrato["valor_mensal"] / self.DIAS_BASE_MULTA
            multa = round(dias_atraso * valor_dia, 2)
            return NotificacaoCreate(
                usuario_id=inquilino_id,
                tipo="atraso",
                mensagem=(
                    f"O contrato do imóvel ID {imovel_id} venceu há "
                    f"{dias_atraso} dia(s). Multa acumulada: R$ {multa:.2f}."
                ),
                imovel_id=imovel_id,
            )

        if 0 ⩽ dias_restantes ⩽ self.DIAS_ALERTA_PRAZO:
            return NotificacaoCreate(
                usuario_id=inquilino_id,
                tipo="prazo_proximo",
                mensagem=(
                    f"Atenção! O contrato do imóvel ID {imovel_id} "
                    f"vence em {dias_restantes} dia(s) "
                    f"({contrato['data_fim_prevista']})."
                ),
                imovel_id=imovel_id,
            )

        return None
```

A.6 Microsserviço de Recomendação

biblioteca/services/recomendacao/models.py 19 linhas

```python
from pydantic import BaseModel


class ImovelRecomendado(BaseModel):
    imovel_id: int
    titulo: str
    tipo: str
    cidade: str
    valor_mensal: float
    score: float
    motivo: str


class PerfilUsuario(BaseModel):
    usuario_id: int
    tipos_favoritos: list[str]
    cidades_favoritas: list[str]
    total_contratos: int
```

biblioteca/services/recomendacao/routes.py 18 linhas

```python
from fastapi import APIRouter

from models import ImovelRecomendado, PerfilUsuario
from service import RecomendacaoService


def criar_router(service: RecomendacaoService) -> APIRouter:
    router = APIRouter(prefix="/recomendacao", tags=["Recomendação"])

    @router.get("/perfil/{usuario_id}", response_model=PerfilUsuario)
    def obter_perfil(usuario_id: int):
        return service.obter_perfil(usuario_id)

    @router.get("/{usuario_id}", response_model=list[ImovelRecomendado])
    def recomendar(usuario_id: int, limite: int = 5):
        return service.recomendar(usuario_id, limite)

    return router
```

biblioteca/services/recomendacao/service.py 146 linhas

```python
from collections import Counter

from models import ImovelRecomendado, PerfilUsuario
from repository import RecomendacaoRepository
from clients import ContratoClient, ImovelClient


class RecomendacaoService:
    """Recomenda imoveis a partir do historico do usuario.

    O algoritmo e o mesmo do acervo de biblioteca, com as preferencias
    remapeadas para o dominio de aluguel: genero virou tipo (apartamento/casa)
    e autor virou cidade. Na Fase 4 este servico e substituido pelo agente,
    entao a pontuacao aqui e deliberadamente simples.
    """

    PONTOS_TIPO = 3
    PONTOS_CIDADE = 2
    PENALIDADE_JA_MOROU = 2
```

```
    PENALIDADE_JA_RECOMENDADO = 1
    TOP_N_PREFERENCIAS = 3

    def __init__(
        self,
        repository: RecomendacaoRepository,
        contratos: ContratoClient,
        imoveis: ImovelClient,
    ):
        self._repo = repository
        self._contratos = contratos
        self._imoveis = imoveis

    def obter_perfil(self, usuario_id: int) -> PerfilUsuario:
        historico = self._contratos.buscar_historico(usuario_id)

        if not historico:
            return PerfilUsuario(
                usuario_id=usuario_id,
                tipos_favoritos=[],
                cidades_favoritas=[],
                total_contratos=0,
            )

        por_id = self._indexar(self._imoveis.buscar_portfolio_completo())
        tipos, cidades = self._coletar_preferencias(historico, por_id)
        return PerfilUsuario(
            usuario_id=usuario_id,
            tipos_favoritos=self._mais_frequentes(tipos),
            cidades_favoritas=self._mais_frequentes(cidades),
            total_contratos=len(historico),
        )

    def recomendar(self, usuario_id: int, limite: int = 5) -> list[ImovelRecomendado]:
        historico = self._contratos.buscar_historico(usuario_id)
        portfolio = self._imoveis.buscar_portfolio_completo()

        em_aberto = {c["imovel_id"] for c in historico if c.get("status") != "encerrado"}
        ja_morou = {c["imovel_id"] for c in historico} - em_aberto
        tipos, cidades = self._coletar_preferencias(historico, self._indexar(portfolio))
        top_tipos = set(self._mais_frequentes(tipos))
        top_cidades = set(self._mais_frequentes(cidades))
        ja_recomendados = self._repo.listar_recomendados(usuario_id)

        recomendacoes = []
        for imovel in portfolio:
            if imovel["id"] in em_aberto:
                continue
            recomendado = self._pontuar(
                imovel, top_tipos, top_cidades,
                ja_morou, ja_recomendados,
            )
            if recomendado is not None:
                recomendacoes.append(recomendado)

        recomendacoes.sort(key=lambda r: r.score, reverse=True)
        resultado = recomendacoes[:limite]

        self._repo.registrar_recomendacoes(
            usuario_id, [rec.imovel_id for rec in resultado]
        )
        return resultado

    @staticmethod
    def _indexar(portfolio: list[dict]) -> dict:
        return {imovel["id"]: imovel for imovel in portfolio}

    def _coletar_preferencias(
        self, historico: list[dict], por_id: dict
```

```
    ) -> tuple[list, list]:
        """Tipo e cidade de cada imóvel do histórico.

        Sai do portfólio já carregado, não de uma chamada HTTP por contrato.
        """
        tipos, cidades = [], []
        for contrato in historico:
            imovel = por_id.get(contrato["imovel_id"])
            if imovel:
                tipos.append(imovel["tipo"])
                cidades.append(imovel["cidade"])
        return tipos, cidades

    def _mais_frequentes(self, valores: list[str]) -> list[str]:
        return [v for v, _ in Counter(valores).most_common(self.TOP_N_PREFERENCIAS)]

    def _pontuar(
        self,
        imovel: dict,
        top_tipos: set,
        top_cidades: set,
        ja_morou: set,
        ja_recomendados: set,
    ) -> ImovelRecomendado | None:
        if not imovel.get("disponivel", False):
            return None

        score = 0.0
        motivo = []

        if imovel["tipo"] in top_tipos:
            score += self.PONTOS_TIPO
            motivo.append(f"tipo '{imovel['tipo']}' está entre seus preferidos")

        if imovel["cidade"] in top_cidades:
            score += self.PONTOS_CIDADE
            motivo.append(f"cidade '{imovel['cidade']}' está entre suas preferidas")

        if imovel["id"] in ja_morou:
            score -= self.PENALIDADE_JA_MOROU
            motivo.append("você já morou aqui")

        if imovel["id"] in ja_recomendados:
            score -= self.PENALIDADE_JA_RECOMENDADO

        if score <= 0:
            return None

        return ImovelRecomendado(
            imovel_id=imovel["id"],
            titulo=imovel["titulo"],
            tipo=imovel["tipo"],
            cidade=imovel["cidade"],
            valor_mensal=imovel["valor_mensal"],
            score=score,
            motivo=" e ".join(motivo) if motivo else "recomendação geral",
        )
```

biblioteca/services/recomendacao/clients.py 37 linhas

```
import httpx

from exceptions import ServicoIndisponivel


class ContratoClient:

    def __init__(self, base_url: str, timeout: float = 5.0):
```

```
        self._base_url = base_url.rstrip("/")
        self._timeout = timeout

    def buscar_historico(self, inquilino_id: int) -> list[dict]:
        try:
            resp = httpx.get(
                f"{self._base_url}/contratos/",
                params={"inquilino_id": inquilino_id},
                timeout=self._timeout,
            )
            resp.raise_for_status()
            return resp.json()
        except httpx.RequestError:
            raise ServicoIndisponivel("Serviço de Contratos indisponível")


class ImovelClient:

    def __init__(self, base_url: str, timeout: float = 5.0):
        self._base_url = base_url.rstrip("/")
        self._timeout = timeout

    def buscar_portfolio_completo(self) -> list[dict]:
        try:
            resp = httpx.get(f"{self._base_url}/imoveis/", timeout=self._timeout)
            resp.raise_for_status()
            return resp.json()
        except httpx.RequestError:
            raise ServicoIndisponivel("Serviço de Imóveis indisponível")
```

A.7 Microsserviço de Agente

biblioteca/services/agente/routes.py 39 linhas

```python
from typing import Optional

from fastapi import APIRouter, Depends, Header

from models import Pergunta, Resposta, PedidoAviso, Aviso
from agente import Agente


def identificar(
    x_usuario_id: Optional[int] = Header(None),
    x_usuario_tipo: Optional[str] = Header(None),
) -> Optional[dict]:
    if x_usuario_id is None:
        return None
    return {"id": x_usuario_id, "tipo": x_usuario_tipo or ""}


def criar_router(agente: Agente) -> APIRouter:
    router = APIRouter(prefix="/agente", tags=["Agente"])

    @router.post("/perguntar", response_model=Resposta)
    def perguntar(pergunta: Pergunta, chamador=Depends(identificar)):
        return agente.perguntar(pergunta.texto, pergunta.inquilino_id, chamador)

    @router.post("/redigir-aviso", response_model=Aviso)
    def redigir_aviso(pedido: PedidoAviso):
        return {"mensagem": agente.redigir_aviso(pedido.fatos), "modelo": agente.modelo}

    @router.get("/status")
    def status():
        return {
            "credencial_configurada": agente.tem_credencial(),
            "provedor": agente.provedor,
            "modelo": agente.modelo,
            "ferramentas": ["buscar_imoveis", "detalhar_imovel", "listar_contratos"],
        }

    return router
```

biblioteca/services/agente/agente.py 114 linhas

```python
"""O agente: recebe pergunta em linguagem natural e responde usando as tools.

Quem fala com o modelo e o provedor escolhido em `provedores.py` (Gemini ou
Claude); aqui ficam so as instrucoes e as regras que valem para os dois.
"""
from ferramentas import FERRAMENTAS, chamador_atual
from exceptions import AgenteIndisponivel, RegraDeNegocio
from provedores import escolher_provedor

INSTRUCOES = """Você é o assistente de uma imobiliária que aluga apartamentos e
casas em Maceió, Recife e João Pessoa.

Responda SEMPRE em português do Brasil, de forma curta e direta.

Regras:
- Use as ferramentas para obter dados. Nunca invente imóvel, preço, endereço ou
  contrato: se não veio de uma ferramenta, você não sabe.
- Ao sugerir imóveis, cite o título, a cidade e o aluguel mensal de cada um.
- Se a busca não retornar nada, diga isso claramente e sugira afrouxar um
  critério (outra cidade, mais orçamento, menos quartos). Não ofereça imóvel que
  não apareceu no resultado.
```

```
- Se a pessoa não disser a cidade ou o orçamento, busque assim mesmo com o que
  ela deu, em vez de ficar perguntando.
- Você não registra, altera nem encerra contratos. Se pedirem isso, explique que
  a contratação é feita na tela de Contratos.
"""


INSTRUCOES_AVISO = """Você escreve avisos curtos de uma imobiliária para o
inquilino, em português do Brasil.

Regras:
- Use SOMENTE os fatos que vierem na mensagem. Não invente data, valor, endereço
  nem nome.
- NÃO faça contas. Não calcule multa, total devido, nem diferença de dias. Se um
  valor não foi informado, não mencione valor.
- Dois a quatro períodos, tom cordial e direto, sem saudação formal nem
  assinatura.
- Termine orientando a pessoa a procurar a imobiliária para renovar o contrato
  ou agendar a desocupação.
- Responda apenas com o texto do aviso, sem aspas e sem comentários.
"""


class Agente:
    """Encapsula o provedor de modelo e as regras de quem pergunta.

    O provedor e escolhido ao criar o agente, mas a credencial so e exigida
    na hora da pergunta: o servico precisa subir e responder /health mesmo
    sem chave configurada.
    """

    def __init__(self, provedor=None):
        self._provedor = provedor or escolher_provedor()

    @property
    def provedor(self) -> str:
        return self._provedor.nome

    @property
    def modelo(self) -> str:
        return self._provedor.modelo

    def tem_credencial(self) -> bool:
        return self._provedor.disponivel()

    def perguntar(
        self, pergunta: str, inquilino_id: int = None, chamador: dict = None
    ) -> dict:
        texto = (pergunta or "").strip()
        if not texto:
            raise RegraDeNegocio("A pergunta não pode estar vazia")

        if chamador is not None:
            inquilino_id = chamador["id"]

        contexto = texto
        if inquilino_id is not None:
            contexto = (
                f"[inquilino_id do usuário autenticado: {inquilino_id}]" + chr(10) + texto
            )

        marca = chamador_atual.set(chamador)
        try:
            resultado = self._provedor.responder(INSTRUCOES, contexto, FERRAMENTAS)
        finally:
            chamador_atual.reset(marca)

        return {
            "resposta": resultado["resposta"] or "Não consegui formular uma resposta.",
```

```
            "ferramentas_usadas": resultado["ferramentas_usadas"],
            "modelo": self.modelo,
            "truncado": resultado["truncado"],
        }

    def redigir_aviso(self, fatos: dict) -> str:
        """Redige o aviso de vencimento a partir de fatos já apurados.

        Sem ferramentas de propósito: quem levantou os contratos foi a aplicação,
        usando os componentes. O modelo só escreve o texto, não busca dados e
        não faz contas. Isso mantém a regra de multa num lugar só (o serviço de
        contratos) e tira do modelo a chance de inventar número.
        """
        if not fatos:
            raise RegraDeNegocio("Nenhum fato informado para o aviso")

        linhas = [f"- {chave}: {valor}" for chave, valor in fatos.items()]
        conteudo = "Fatos do contrato:" + chr(10) + chr(10).join(linhas)

        texto = self._provedor.redigir(INSTRUCOES_AVISO, conteudo)
        if not texto:
            raise AgenteIndisponivel("O modelo devolveu aviso vazio")
        return texto
```

biblioteca/services/agente/provedores.py 217 linhas

```
"""Provedores de modelo do agente: Gemini (Google) e Claude (Anthropic).

Os dois expoem a mesma interface (`disponivel`, `responder` e `redigir`), entao
o agente nao sabe qual esta usando. A escolha vem de AGENTE_PROVEDOR
("gemini" ou "anthropic"); sem ela, vale o primeiro que tiver credencial no
ambiente, com o Gemini na frente, porque tem nivel gratuito.

Os SDKs sao importados so na hora do uso: o servico sobe e responde /health
mesmo sem nenhum deles configurado.
"""
import os

import httpx

from exceptions import AgenteIndisponivel, RegraDeNegocio

MAX_ITERACOES = 8
ESFORCO = os.getenv("AGENTE_ESFORCO", "medium")

SEM_CREDENCIAL = (
    "Nenhuma credencial de modelo encontrada. Defina GEMINI_API_KEY (chave "
    "gratuita em aistudio.google.com) ou ANTHROPIC_API_KEY no ambiente e "
    "reinicie o serviço."
)


class ProvedorGemini:
    """Gemini pela API do Google AI Studio, com chamada automatica de funcoes.

    O SDK executa as ferramentas sozinho, na mesma thread, e devolve o
    historico das chamadas; e dele que sai a lista de ferramentas usadas.
    """

    nome = "gemini"

    def __init__(self):
        self.modelo = os.getenv("AGENTE_MODELO_GEMINI", "gemini-3.8-flash")
        self._cliente = None

    def disponivel(self) -> bool:
        return bool(os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY"))

```

```python
    def _obter_cliente(self):
        if not self.disponivel():
            raise AgenteIndisponivel(SEM_CREDENCIAL)
        if self._cliente is None:
            from google import genai
            self._cliente = genai.Client()
        return self._cliente

    def _gerar(self, conteudo: str, config):
        from google.genai import errors
        try:
            return self._obter_cliente().models.generate_content(
                model=self.modelo, contents=conteudo, config=config
            )
        except errors.APIError as erro:
            raise AgenteIndisponivel(
                f"A API do Gemini recusou a chamada ({erro.code}): {erro.message}"
            )
        except httpx.TransportError as erro:
            raise AgenteIndisponivel(f"Falha ao falar com a API do Gemini: {erro}")

    def _pensamento(self, nivel: str):
        from google.genai import types
        return types.ThinkingConfig(thinking_level=types.ThinkingLevel(nivel.upper()))

    def responder(self, instrucoes: str, texto: str, ferramentas: list) -> dict:
        from google.genai import types
        config = types.GenerateContentConfig(
            system_instruction=instrucoes,
            tools=ferramentas,
            automatic_function_calling=types.AutomaticFunctionCallingConfig(
                maximum_remote_calls=MAX_ITERACOES
            ),
            thinking_config=self._pensamento(ESFORCO),
        )
        resposta = self._gerar(texto, config)

        feedback = resposta.prompt_feedback
        if feedback is not None and feedback.block_reason:
            raise RegraDeNegocio(
                f"O modelo recusou responder a esta pergunta ({feedback.block_reason})"
            )

        usadas = []
        for conteudo in resposta.automatic_function_calling_history or []:
            for parte in conteudo.parts or []:
                if parte.function_call:
                    usadas.append({
                        "ferramenta": parte.function_call.name,
                        "argumentos": dict(parte.function_call.args or {}),
                    })

        return {
            "resposta": (resposta.text or "").strip(),
            "ferramentas_usadas": usadas,
            "truncado": bool(resposta.function_calls),
        }

    def redigir(self, instrucoes: str, conteudo: str) -> str:
        from google.genai import types
        config = types.GenerateContentConfig(
            system_instruction=instrucoes, thinking_config=self._pensamento("low")
        )
        return (self._gerar(conteudo, config).text or "").strip()


class ProvedorAnthropic:
    """Claude pela API da Anthropic, com o Tool Runner do SDK."""

```

```
    nome = "anthropic"
    MAX_TOKENS = 4096

    def __init__(self):
        self.modelo = os.getenv("AGENTE_MODELO", "claude-opus-5-5")
        self._cliente = None

    def _obter_cliente(self):
        if self._cliente is None:
            import anthropic
            cliente = anthropic.Anthropic()
            if not (cliente.api_key or cliente.auth_token or cliente.credentials):
                raise AgenteIndisponivel(SEM_CREDENCIAL)
            self._cliente = cliente
        return self._cliente

    def disponivel(self) -> bool:
        try:
            self._obter_cliente()
            return True
        except (AgenteIndisponivel, ImportError):
            return False

    def _chamar(self, funcao):
        import anthropic
        try:
            return funcao()
        except anthropic.APIStatusError as erro:
            raise AgenteIndisponivel(
                f"A API da Anthropic recusou a chamada ({erro.status_code}): {erro}"
            )
        except anthropic.APIConnectionError as erro:
            raise AgenteIndisponivel(f"Falha ao falar com a API da Anthropic: {erro}")
        except TypeError as erro:
            if "authentication" in str(erro).lower():
                raise AgenteIndisponivel(SEM_CREDENCIAL)
            raise

    def responder(self, instrucoes: str, texto: str, ferramentas: list) -> dict:
        from anthropic import beta_tool
        cliente = self._obter_cliente()

        def executar():
            runner = cliente.beta.messages.tool_runner(
                model=self.modelo,
                max_tokens=self.MAX_TOKENS,
                system=instrucoes,
                output_config={"effort": ESFORCO},
                tools=[beta_tool(f) for f in ferramentas],
                messages=[{"role": "user", "content": texto}],
            )
            usadas, ultima, iteracoes = [], None, 0
            for mensagem in runner:
                ultima = mensagem
                iteracoes += 1
                for bloco in mensagem.content:
                    if bloco.type == "tool_use":
                        usadas.append({"ferramenta": bloco.name, "argumentos": bloco.input})
                if iteracoes >= MAX_ITERACOES:
                    break
            return usadas, ultima, iteracoes

        usadas, ultima, iteracoes = self._chamar(executar)
        if ultima is None:
            raise AgenteIndisponivel("O modelo não devolveu resposta")
        if ultima.stop_reason == "refusal":
            motivo = getattr(ultima, "stop_details", None)
            raise RegraDeNegocio(
                "O modelo recusou responder a esta pergunta"
```

```
                + (f" ({motivo.category})" if motivo else "")
            )
        return {
            "resposta": "".join(b.text for b in ultima.content if b.type == "text").strip(),
            "ferramentas_usadas": usadas,
            "truncado": iteracoes >= MAX_ITERACOES and ultima.stop_reason == "tool_use",
        }

    def redigir(self, instrucoes: str, conteudo: str) -> str:
        cliente = self._obter_cliente()
        resposta = self._chamar(lambda: cliente.messages.create(
            model=self.modelo,
            max_tokens=600,
            system=instrucoes,
            output_config={"effort": "low"},
            messages=[{"role": "user", "content": conteudo}],
        ))
        if resposta.stop_reason == "refusal":
            raise RegraDeNegocio("O modelo recusou redigir este aviso")
        return "".join(b.text for b in resposta.content if b.type == "text").strip()


PROVEDORES = {"gemini": ProvedorGemini, "anthropic": ProvedorAnthropic}


def escolher_provedor():
    """AGENTE_PROVEDOR manda; sem ele, o primeiro com credencial, Gemini antes."""
    escolhido = os.getenv("AGENTE_PROVEDOR", "").strip().lower()
    if escolhido:
        if escolhido not in PROVEDORES:
            raise ValueError(
                f"AGENTE_PROVEDOR={escolhido!r} não existe; use gemini ou anthropic"
            )
        return PROVEDORES[escolhido]()
    candidatos = [ProvedorGemini(), ProvedorAnthropic()]
    return next((p for p in candidatos if p.disponivel()), candidatos[0])
```

biblioteca/services/agente/ferramentas.py 109 linhas

```
"""Ferramentas do agente.

Este modulo e o argumento de arquitetura do projeto: o agente nao fala com os
microsservicos por conta propria, ele usa a MESMA camada de componentes que as
aplicacoes convencionais do framework (`framework/componentes.py`). Cada
ferramenta aqui e uma casca fina sobre um metodo de `IComponenteImovel` ou
`IComponenteContrato` -- a assinatura do componente ja e, na pratica, a
assinatura de uma tool.

As ferramentas sao todas de LEITURA. Registrar contrato e acao com efeito
colateral e dinheiro envolvido; isso nao fica a cargo do modelo nesta fase.

Sao funcoes Python comuns, sem depender de SDK: cada provedor de modelo
(`provedores.py`) converte a assinatura e a docstring para o formato da sua
API. Por isso as docstrings seguem o formato Args, que os dois entendem.
"""
import json
import os
import sys
from contextvars import ContextVar

sys.path.insert(
    0, os.path.join(os.path.dirname(__file__), "..", "..", "framework")
)

from componentes import (
    ComponenteImovelHTTP,
    ComponenteContratoHTTP,
)
```

```python

imoveis = ComponenteImovelHTTP()
contratos = ComponenteContratoHTTP()

chamador_atual: ContextVar = ContextVar("chamador_atual", default=None)


def _json(valor) -> str:
    """Resultado de tool precisa ser texto; JSON e o formato mais legivel."""
    return json.dumps(valor, ensure_ascii=False, default=str)


def buscar_imoveis(
    cidade: str = "",
    tipo: str = "",
    quartos_min: int = 0,
    valor_max: float = 0.0,
    valor_min: float = 0.0,
) -> str:
    """Busca imoveis DISPONIVEIS para aluguel no portfolio.

    Use esta ferramenta sempre que a pessoa descrever o que procura. Todos os
    filtros sao opcionais: envie apenas os que a pessoa mencionou. Um filtro
    omitido nao restringe a busca.

    Args:
        cidade: Nome da cidade, por exemplo "Maceio", "Recife", "Joao Pessoa".
        tipo: "apartamento" ou "casa". Deixe vazio se a pessoa nao disse.
        quartos_min: Numero minimo de quartos. 0 significa sem minimo.
        valor_max: Aluguel mensal maximo em reais. 0 significa sem teto.
        valor_min: Aluguel mensal minimo em reais. 0 significa sem piso.
    """
    filtros = {"disponivel": True}
    if cidade:
        filtros["cidade"] = cidade
    if tipo:
        filtros["tipo"] = tipo
    if quartos_min:
        filtros["quartos_min"] = quartos_min
    if valor_max:
        filtros["valor_max"] = valor_max
    if valor_min:
        filtros["valor_min"] = valor_min

    encontrados = imoveis.buscar_imoveis(filtros)
    return _json({"total": len(encontrados), "imoveis": encontrados})


def detalhar_imovel(imovel_id: int) -> str:
    """Devolve os dados completos de um imovel especifico pelo id.

    Use quando a pessoa perguntar sobre um imovel que ja apareceu na conversa.

    Args:
        imovel_id: Identificador do imovel.
    """
    portfolio = imoveis.buscar_imoveis({})
    imovel = next((i for i in portfolio if i["id"] == imovel_id), None)
    if imovel is None:
        return _json({"erro": f"Imovel {imovel_id} nao existe no portfolio"})
    return _json(imovel)


def listar_contratos(inquilino_id: int) -> str:
    """Lista os contratos de aluguel de um inquilino, em vigor e encerrados.

    Use quando a pessoa perguntar sobre os proprios contratos, o que ja alugou,
    prazos ou multas.

```

```
    Args:
        inquilino_id: Identificador do inquilino.
    """
    chamador = chamador_atual.get()
    if chamador and chamador["tipo"] ≠ "admin" and inquilino_id ≠ chamador["id"]:
        return _json({"erro": "Sem permissão para ver contratos de outro inquilino"})
    return _json(contratos.listar_contratos(inquilino_id))


FERRAMENTAS = [buscar_imoveis, detalhar_imovel, listar_contratos]
```

A.8 Componentes de software e framework

biblioteca/framework/interfaces.py 101 linhas

```python
from abc import ABC, abstractmethod


class IComponente(ABC):

    @abstractmethod
    def inicializar(self) -> None:
        pass

    @abstractmethod
    def finalizar(self) -> None:
        pass

    @abstractmethod
    def get_nome(self) -> str:
        pass


class IComponenteImovel(IComponente):

    @abstractmethod
    def buscar_imoveis(self, filtros: dict) -> list[dict]:
        pass

    @abstractmethod
    def cadastrar_imovel(self, dados: dict) -> dict:
        pass

    @abstractmethod
    def definir_disponibilidade(self, imovel_id: int, disponivel: bool) -> None:
        pass


class IComponenteUsuario(IComponente):

    @abstractmethod
    def registrar(self, dados: dict) -> dict:
        pass

    @abstractmethod
    def autenticar(self, email: str, senha: str) -> dict | None:
        pass

    @abstractmethod
    def buscar_usuario(self, usuario_id: int) -> dict | None:
        pass


class IComponenteContrato(IComponente):

    @abstractmethod
    def registrar_contrato(self, inquilino_id: int, imovel_id: int) -> dict:
        pass

    @abstractmethod
    def encerrar_contrato(self, contrato_id: int) -> dict:
        pass

    @abstractmethod
    def listar_contratos(self, inquilino_id: int = None) -> list[dict]:
        """inquilino_id ausente lista os contratos de todos."""
        pass

```

```python

class IComponenteNotificacao(IComponente):

    @abstractmethod
    def enviar(self, usuario_id: int, tipo: str, mensagem: str, imovel_id: int = None) -> dict:
        pass

    @abstractmethod
    def listar(self, usuario_id: int) -> list[dict]:
        pass


class IComponenteAgente(IComponente):
    """O agente exposto como componente.

    Simetria que vale notar: o agente CONSOME componentes (as ferramentas dele)
    e tambem E um componente. Assim uma app do framework orquestra o agente
    pelo mesmo contrato que usa para qualquer outro servico.
    """

    @abstractmethod
    def perguntar(self, texto: str, inquilino_id: int = None) -> dict:
        pass

    @abstractmethod
    def redigir_aviso(self, fatos: dict) -> str:
        pass


class IComponenteRecomendacao(IComponente):

    @abstractmethod
    def recomendar(self, usuario_id: int, limite: int) -> list[dict]:
        pass

    @abstractmethod
    def obter_perfil(self, usuario_id: int) -> dict:
        pass
```

biblioteca/framework/componentes.py 226 linhas

```python
import os

import httpx
from interfaces import (
    IComponenteImovel,
    IComponenteUsuario,
    IComponenteContrato,
    IComponenteNotificacao,
    IComponenteAgente,
    IComponenteRecomendacao,
)


URL_IMOVEIS      = os.getenv("IMOVEIS_URL",      "http://127.0.0.1:8001")
URL_USUARIOS     = os.getenv("USUARIOS_URL",     "http://127.0.0.1:8002")
URL_CONTRATOS    = os.getenv("CONTRATOS_URL",    "http://127.0.0.1:8003")
URL_NOTIFICACOES = os.getenv("NOTIFICACOES_URL", "http://127.0.0.1:8004")
URL_RECOMENDACAO = os.getenv("RECOMENDACAO_URL", "http://127.0.0.1:8005")
URL_AGENTE       = os.getenv("AGENTE_URL",       "http://127.0.0.1:8006")


class ComponenteImovelHTTP(IComponenteImovel):

    def __init__(self, base_url: str = URL_IMOVEIS):
        self._url    = base_url
        self._client = None

```

```python
    def inicializar(self) -> None:
        self._client = httpx.Client(timeout=30.0)

    def finalizar(self) -> None:
        if self._client:
            self._client.close()

    def get_nome(self) -> str:
        return "imoveis"

    def buscar_imoveis(self, filtros: dict = None) -> list[dict]:
        params = filtros or {}
        resp = self._client.get(f"{self._url}/imoveis/", params=params)
        resp.raise_for_status()
        return resp.json()

    def cadastrar_imovel(self, dados: dict) -> dict:
        resp = self._client.post(f"{self._url}/imoveis/", json=dados)
        resp.raise_for_status()
        return resp.json()

    def definir_disponibilidade(self, imovel_id: int, disponivel: bool) -> None:
        resp = self._client.patch(
            f"{self._url}/imoveis/{imovel_id}/disponibilidade",
            params={"disponivel": disponivel},
        )
        resp.raise_for_status()


class ComponenteUsuarioHTTP(IComponenteUsuario):

    def __init__(self, base_url: str = URL_USUARIOS):
        self._url    = base_url
        self._client = None

    def inicializar(self) -> None:
        self._client = httpx.Client(timeout=30.0)

    def finalizar(self) -> None:
        if self._client:
            self._client.close()

    def get_nome(self) -> str:
        return "usuarios"

    def registrar(self, dados: dict) -> dict:
        resp = self._client.post(f"{self._url}/usuarios/registro", json=dados)
        resp.raise_for_status()
        return resp.json()

    def autenticar(self, email: str, senha: str) -> dict | None:
        resp = self._client.post(
            f"{self._url}/usuarios/login",
            json={"email": email, "senha": senha},
        )
        if resp.status_code == 401:
            return None
        resp.raise_for_status()
        return resp.json()

    def buscar_usuario(self, usuario_id: int) -> dict | None:
        resp = self._client.get(f"{self._url}/usuarios/{usuario_id}")
        if resp.status_code == 404:
            return None
        resp.raise_for_status()
        return resp.json()


class ComponenteContratoHTTP(IComponenteContrato):
```

```python

    def __init__(self, base_url: str = URL_CONTRATOS):
        self._url    = base_url
        self._client = None

    def inicializar(self) -> None:
        self._client = httpx.Client(timeout=30.0)

    def finalizar(self) -> None:
        if self._client:
            self._client.close()

    def get_nome(self) -> str:
        return "contratos"

    def registrar_contrato(self, inquilino_id: int, imovel_id: int) -> dict:
        resp = self._client.post(
            f"{self._url}/contratos/",
            json={"inquilino_id": inquilino_id, "imovel_id": imovel_id},
        )
        resp.raise_for_status()
        return resp.json()

    def encerrar_contrato(self, contrato_id: int) -> dict:
        resp = self._client.post(f"{self._url}/contratos/{contrato_id}/encerrar")
        resp.raise_for_status()
        return resp.json()

    def listar_contratos(self, inquilino_id: int = None) -> list[dict]:
        params = {} if inquilino_id is None else {"inquilino_id": inquilino_id}
        resp = self._client.get(f"{self._url}/contratos/", params=params)
        resp.raise_for_status()
        return resp.json()


class ComponenteNotificacaoHTTP(IComponenteNotificacao):

    def __init__(self, base_url: str = URL_NOTIFICACOES):
        self._url    = base_url
        self._client = None

    def inicializar(self) -> None:
        self._client = httpx.Client(timeout=30.0)

    def finalizar(self) -> None:
        if self._client:
            self._client.close()

    def get_nome(self) -> str:
        return "notificacoes"

    def enviar(self, usuario_id: int, tipo: str, mensagem: str, imovel_id: int = None) -> dict:
        resp = self._client.post(
            f"{self._url}/notificacoes/",
            json={
                "usuario_id": usuario_id,
                "tipo": tipo,
                "mensagem": mensagem,
                "imovel_id": imovel_id,
            },
        )
        resp.raise_for_status()
        return resp.json()

    def listar(self, usuario_id: int) -> list[dict]:
        resp = self._client.get(f"{self._url}/notificacoes/usuario/{usuario_id}")
        resp.raise_for_status()
        return resp.json()

```

```python

class ComponenteRecomendacaoHTTP(IComponenteRecomendacao):

    def __init__(self, base_url: str = URL_RECOMENDACAO):
        self._url    = base_url
        self._client = None

    def inicializar(self) -> None:
        self._client = httpx.Client(timeout=30.0)

    def finalizar(self) -> None:
        if self._client:
            self._client.close()

    def get_nome(self) -> str:
        return "recomendacao"

    def recomendar(self, usuario_id: int, limite: int = 5) -> list[dict]:
        resp = self._client.get(
            f"{self._url}/recomendacao/{usuario_id}",
            params={"limite": limite},
        )
        resp.raise_for_status()
        return resp.json()

    def obter_perfil(self, usuario_id: int) -> dict:
        resp = self._client.get(f"{self._url}/recomendacao/perfil/{usuario_id}")
        resp.raise_for_status()
        return resp.json()

class ComponenteAgenteHTTP(IComponenteAgente):

    def __init__(self, base_url: str = URL_AGENTE):
        self._url    = base_url
        self._client = None

    def inicializar(self) -> None:
        self._client = httpx.Client(timeout=120.0)

    def finalizar(self) -> None:
        if self._client:
            self._client.close()

    def get_nome(self) -> str:
        return "agente"

    def perguntar(self, texto: str, inquilino_id: int = None) -> dict:
        corpo = {"texto": texto}
        if inquilino_id is not None:
            corpo["inquilino_id"] = inquilino_id
        resp = self._client.post(f"{self._url}/agente/perguntar", json=corpo)
        resp.raise_for_status()
        return resp.json()

    def redigir_aviso(self, fatos: dict) -> str:
        resp = self._client.post(
            f"{self._url}/agente/redigir-aviso", json={"fatos": fatos}
        )
        resp.raise_for_status()
        return resp.json()["mensagem"]
```

biblioteca/framework/framework.py 104 linhas

```python
from abc import ABC, abstractmethod
from datetime import datetime
from typing import Any
from interfaces import IComponente

```

```python

class FrameworkBiblioteca(ABC):

    def __init__(self):
        self._componentes: dict[str, IComponente] = {}
        self._log: list[dict]                     = []
        self._inicializado: bool                  = False

    def executar_operacao(self, contexto: dict = None) -> Any:
        if contexto is None:
            contexto = {}

        resultado = None
        inicio = datetime.now()

        try:
            self._inicializar_componentes()
            self._validar_pre_condicoes(contexto)
            self.pre_processar(contexto)
            resultado = self.executar_logica(contexto)
            self.pos_processar(contexto, resultado)

        except Exception as erro:
            resultado = self.tratar_erro(contexto, erro)

        finally:
            duracao = (datetime.now() - inicio).total_seconds()
            self._registrar_log(contexto, resultado, duracao)
            self._finalizar_componentes()

        return resultado

    def _inicializar_componentes(self) -> None:
        if not self._inicializado:
            self.configurar_componentes()
            self._inicializado = True

        for nome, componente in self._componentes.items():
            componente.inicializar()

    def _validar_pre_condicoes(self, contexto: dict) -> None:
        campos_obrigatorios = self.get_campos_obrigatorios()
        for campo in campos_obrigatorios:
            if campo not in contexto:
                raise ValueError(
                    f"Campo obrigatório ausente no contexto: '{campo}'. "
                    f"Campos exigidos: {campos_obrigatorios}"
                )

    def _registrar_log(self, contexto: dict, resultado: Any, duracao: float) -> None:
        entrada = {
            "timestamp": datetime.now().isoformat(),
            "operacao": self.__class__.__name__,
            "duracao_segundos": round(duracao, 4),
            "contexto_resumo": {k: v for k, v in contexto.items() if k != "senha"},
            "status": "erro" if isinstance(resultado, dict) and resultado.get("erro") else "ok",
        }
        self._log.append(entrada)

    def _finalizar_componentes(self) -> None:
        for componente in self._componentes.values():
            try:
                componente.finalizar()
            except Exception:
                pass

    def registrar_componente(self, nome: str, componente: IComponente) -> None:
        self._componentes[nome] = componente

```

```
    def get_componente(self, nome: str) -> IComponente:
        if nome not in self._componentes:
            raise KeyError(
                f"Componente '{nome}' não registrado. "
                f"Registre-o em configurar_componentes() via registrar_componente()."
            )
        return self._componentes[nome]

    def get_log(self) -> list[dict]:
        return self._log

    @abstractmethod
    def configurar_componentes(self) -> None:
        pass

    @abstractmethod
    def executar_logica(self, contexto: dict) -> Any:
        pass

    def pre_processar(self, contexto: dict) -> None:
        pass

    def pos_processar(self, contexto: dict, resultado: Any) -> None:
        pass

    def tratar_erro(self, contexto: dict, erro: Exception) -> dict:
        return {"erro": True, "mensagem": str(erro), "tipo": type(erro).__name__}

    def get_campos_obrigatorios(self) -> list[str]:
        return []
```

A.9 Aplicação derivada do framework

biblioteca/apps/app_contrato.py 102 linhas

```python
import sys
import os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..', 'framework'))

from framework import FrameworkBiblioteca
from componentes import (
    ComponenteImovelHTTP,
    ComponenteContratoHTTP,
    ComponenteNotificacaoHTTP,
)


class AppContrato(FrameworkBiblioteca):

    def configurar_componentes(self) -> None:
        self.registrar_componente("imoveis",      ComponenteImovelHTTP())
        self.registrar_componente("contratos",    ComponenteContratoHTTP())
        self.registrar_componente("notificacoes", ComponenteNotificacaoHTTP())

    def get_campos_obrigatorios(self) -> list[str]:
        return ["inquilino_id", "imovel_id"]

    def pre_processar(self, contexto: dict) -> None:
        imoveis = self.get_componente("imoveis")
        imovel_id = contexto["imovel_id"]

        portfolio = imoveis.buscar_imoveis({})
        imovel = next((i for i in portfolio if i["id"] == imovel_id), None)

        if not imovel:
            raise ValueError(f"Imóvel ID {imovel_id} não encontrado no portfólio")

        if not imovel["disponivel"]:
            raise ValueError(
                f"Imóvel '{imovel['titulo']}' já possui contrato em aberto"
            )

        contexto["titulo_imovel"] = imovel["titulo"]
        contexto["valor_mensal"] = imovel["valor_mensal"]

    def executar_logica(self, contexto: dict) -> dict:
        contratos = self.get_componente("contratos")
        return contratos.registrar_contrato(
            contexto["inquilino_id"],
            contexto["imovel_id"],
        )

    def pos_processar(self, contexto: dict, resultado: dict) -> None:
        notificacoes = self.get_componente("notificacoes")
        titulo = contexto.get("titulo_imovel", f"ID {contexto['imovel_id']}")
        fim = resultado.get("data_fim_prevista", "-")
        valor = resultado.get("valor_mensal", contexto.get("valor_mensal", 0))

        notificacoes.enviar(
            usuario_id=contexto["inquilino_id"],
            tipo="contrato_confirmado",
            mensagem=(
                f"Contrato confirmado! Imóvel: '{titulo}'. "
                f"Aluguel mensal: R$ {valor:.2f}. Vigência até {fim}. "
                f"Atraso na desocupação gera multa diária de 1/30 do aluguel."
            ),
            imovel_id=contexto["imovel_id"],
        )
```

```python

    def tratar_erro(self, contexto: dict, erro: Exception) -> dict:
        try:
            notificacoes = self.get_componente("notificacoes")
            notificacoes.enviar(
                usuario_id=contexto.get("inquilino_id", 0),
                tipo="atraso",
                mensagem=f"Não foi possível registrar o contrato: {str(erro)}",
                imovel_id=contexto.get("imovel_id"),
            )
        except Exception:
            pass

        return {"erro": True, "mensagem": str(erro), "tipo": type(erro).__name__}


if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser(description="App 1 - Contrato com Notificação")
    parser.add_argument("--inquilino", type=int, default=1, help="ID do inquilino")
    parser.add_argument("--imovel",    type=int, required=True, help="ID do imóvel")
    args = parser.parse_args()

    print("=" * 60)
    print("APP 1 - Contrato de Aluguel com Notificação Automática")
    print("=" * 60)

    app = AppContrato()
    resultado = app.executar_operacao({
        "inquilino_id": args.inquilino,
        "imovel_id":    args.imovel,
    })

    print(f"\nResultado: {resultado}")
    print(f"\nLog de execução:")
    for entrada in app.get_log():
        print(f"  {entrada}")
```